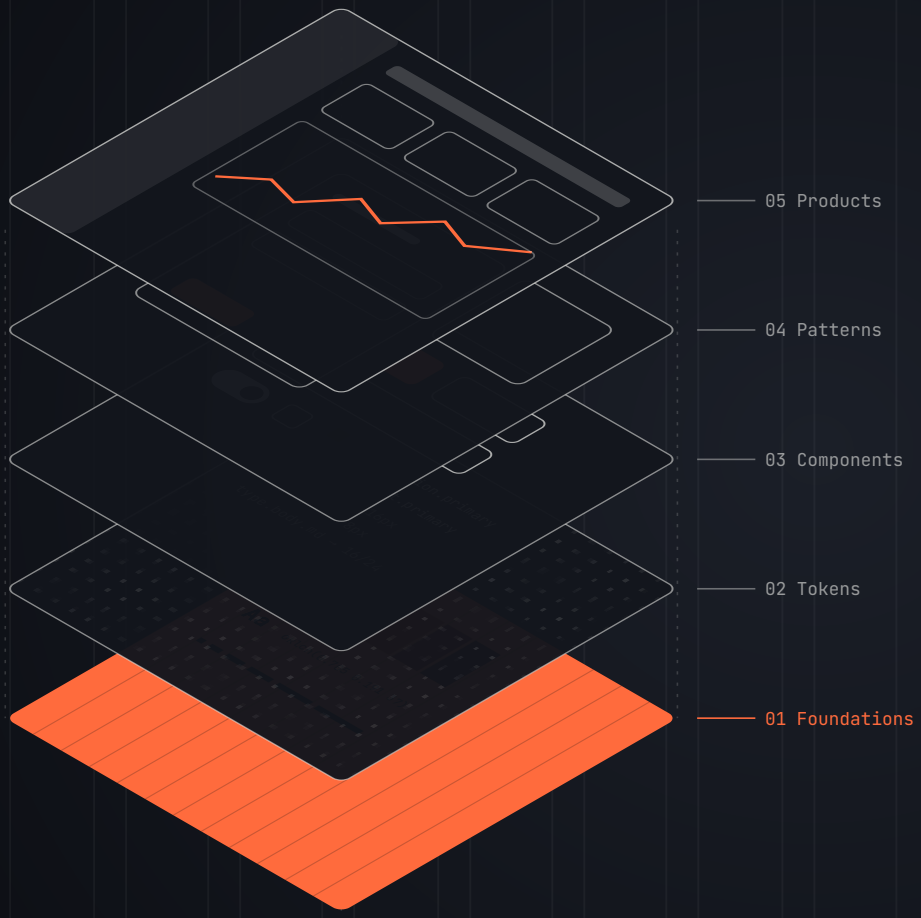


The first layer is the one everything else stands on.

The Design Systems Playbook

Strategy, economics and foundations, for founders, product leaders and the teams who build the system.

AUTHOR
Aniket Pandey

PUBLISHED BY
Layer One, Bangalore

WEB
layerone.co.in

YEAR
2026

ABOUT THIS EDITION

A field guide for the people who decide whether a design system gets built, and how.

This playbook exists because most writing about design systems is either a vendor pitch or a component tutorial. Neither helps a founder decide whether to invest, a design leader plan the first year, or an engineering leader understand what they are signing up to maintain.

It is written by **Aniket Pandey**, who leads **Layer One**, a design systems studio in Bangalore. Aniket has built design systems at IBM (for clients including Cardinal Health and Riyadh Air), led the design system at CARS24 across 10+ products in three countries, and works on design system tooling and governance at JPMorgan Chase.[1] The playbook distils that work, plus the published practice of the most mature public systems, into decisions you can act on.

How to read the evidence

Not every claim in this book is equally certain, and we think you should know which is which. Every claim that matters carries one of four labels:

STANDARD

A normative specification, such as WCAG 2.2. If you claim conformance, it is not optional.

DOCUMENTED

Published research, a case study or official system documentation. Cited, with caveats where the sample is small or vendor-run.

PRACTICE

Layer One's recommendation from practice. Reasoned and tested, but not universal. Disagree when your context differs.

ILLUSTRATIVE

A worked example with stated assumptions. It is not a measured result, so treat the method as the lesson, not the numbers.

We do not invent statistics, case studies or outcomes. Worked examples use **Ledgerly**, a fictional B2B finance product, so you never mistake an illustration for a client result. All colour values, contrast ratios and type scales in the figures are computed by the build script from the data files in the repository, not typed by hand.

EDITION

0.1 — November 2026

Parts 01 and 02. Further parts (components, patterns, governance at scale, tooling) follow.

LICENCE

Free to read and share

Share it freely with attribution to Layer One. Please don't resell it or republish it as your own.

CONTACT

layerone@proton.me

Corrections and disagreements are welcome. We would rather be right than consistent.

CONTENTS

What's inside

Part 01

Understanding design systems: does your company actually need one?

01 What a design system actually is 07

A working definition · Style guide, UI kit, component library, design system · The anatomy of a complete design system · How a design system connects the whole organisation · Why design systems matter more as products scale

02 Who needs a design system, and when 15

The seven questions everyone asks · A maturity model: where you are, and what's next · The decision tree · Signs you have outgrown your current UI infrastructure · Four scenarios · Readiness checklist

03 What a design system is worth 22

Where the value actually comes from · The evidence, honestly · Nine outcomes, and the mechanism behind each · A KPI framework you can baseline this quarter · When value shows up, and when it doesn't

04 The economics of design systems 27

How to read the numbers in this chapter · What determines the investment · Roles and resources · How long it takes · Three budget scenarios · Dedicated team or existing product team? · Ongoing maintenance costs · Calculating ROI · Start small, invest progressively

05 Design system operating models 35

Treat the system as a product · Centralised, federated and hybrid models · Ownership and decision rights · Design and engineering collaboration · Contribution and review · Governance and decision-making · Versioning and release management · Documentation and adoption · Measuring system health · Managing technical and design debt

Part 02

Building the foundations of a design system

06 Understanding design foundations 43

What foundations are · Why foundations matter: blast radius · From decision to token to code

07 Colour systems 47

Colour models: how colour is described · Building a palette · Neutrals: the colour you use most · Colour architecture: primitive, semantic and component tokens · Interactive and state colours · Themes: light, dark, brand and white-label · Practical questions

08 Typography systems 61

Choosing typefaces · Type scale architecture · Roles and hierarchy: a complete product scale · Responsive typography · Accessibility and readability · Typography across product categories · Typography tokens

09 Spacing systems 70

Why spacing systems matter · Choosing a base unit · The spacing scale · Component spacing and layout spacing · Density: compact and comfortable · Responsive spacing and spacing tokens

10 Grid and layout systems 76

Grid fundamentals · Breakpoints and responsive grids · Fluid, fixed and hybrid layouts · Composing a SaaS dashboard · Desktop, mobile and alignment · Layout for complex enterprise applications

11 How foundations work together 82

Back

Reference

– References 87

– About Layer One & the author 91

HOW TO READ THIS PLAYBOOK

You do not need to read it in order.

Part 01 is about decisions: whether to invest, how much, and how to run the thing once it exists. Part 02 is about building: the foundations every component will inherit. Pick the route that matches your job.

Founder or CEO

Do we need this, what does it cost, and when?

02 Who needs one
04 Economics
03 What it's worth
06 Foundations, intro only

Product leader

How will this change delivery and quality?

01 What it is
03 What it's worth
05 Operating models
11 How it fits together

Design leader

How do I plan, staff and govern it?

02 → 05 in order
then 06 → 11
Use the checklists as your plan

Senior designer

How should foundations actually be built?

01 What it is
06 → 11 in order
Chapter 07 in full

Engineering leader

What do we build, version and maintain?

05 Operating models
06 Architecture
07.4 Token architecture
11 Into production

CONVENTIONS

Section numbers look like `07.4` (chapter.section). Values a machine would read (tokens, colours, sizes) are set in `mono`. Colours are written in OKLCH first, with a hex fallback: `oklch(0.486 0.172 35) · #ab2800`.

EVERY CHAPTER OPENS WITH

Read this if, so you can skip what you don't need, and **What you'll leave with**, the three decisions the chapter equips you to make. Each one ends with a practical artefact: a checklist, framework or template.



01

Understanding design systems: does your company actually need one?

Before anyone opens Figma, five questions decide whether a design system succeeds: what it is, whether you need it now, what it is worth, what it costs, and who will run it. This part answers all five.

01 What a design system actually is

02 Who needs one, and when

03 What it's worth

04 The economics

05 Operating models

CHAPTER 01

What a design system actually is

A design system is not a Figma file and it is not a component library. It is a shared agreement about how your product gets made, encoded in tools that design and engineering both use, and kept alive by people whose job it is to keep it alive.

READ THIS IF

You have heard "design system" used for five different things in one meeting, or you need to explain to leadership what you are actually proposing to fund.

WHAT YOU'LL LEAVE WITH

01 A precise, defensible definition you can put in a strategy document.

02 The ability to tell a style guide, UI kit, component library and design system apart, and to know which one you have today.

03 A clear picture of why a system's value grows faster than the organisation using it.

01.1

A working definition

The term is used loosely, and the looseness causes real damage. Leadership approves a "design system" thinking it bought consistency. The design team ships a Figma library. Engineering never adopts it. Eighteen months later the product still has nine kinds of button, and everyone concludes design systems don't work.

So we start with a definition precise enough to plan against:

DEFINITION

A design system is the set of shared decisions, reusable assets and working agreements that lets many people build one coherent product, plus the team and process that keep all three current.

Each of the three parts matters, and most failed systems are missing one of them.

01 — DECISIONS

What we've agreed

Principles, foundations (colour, type, spacing, layout, motion), accessibility targets, content voice. The *why* behind every asset.

02 — ASSETS

What we've built

Design tokens, components in the design tool *and* in code, patterns, templates, icons and the documentation that explains them.

03 — AGREEMENTS

How we work

Ownership, contribution, review, versioning, release and deprecation. The rules for how the first two change over time.

Decisions without assets are a PDF nobody opens. Assets without decisions are a library that encodes whatever inconsistency already existed. Decisions and assets without agreements drift out of date within a year. Layer One's first working rule says it plainly:

> **A design system is an agreement, not a library.**
>
> Rule #1 — Layer One

A short lineage

Organisations have codified visual decisions for decades, in brand manuals and corporate identity standards. What changed in software was that the decisions had to be *executable*. Pattern libraries spread through web teams in the early 2010s. Brad Frost's *Atomic Design* gave teams a shared mental model for composing interfaces from small parts into large ones.[2] Salesforce's design system team popularised the term *design tokens* for named design decisions that can be compiled into any platform's code.[3] In October 2025 the W3C Design Tokens Community Group published the first stable version of a vendor-neutral format for those tokens.[4] Design systems have moved from a design-team initiative to shared product infrastructure, and the standards have caught up.

01.2

Style guide, UI kit, component library, design system

These four terms are not synonyms. Each is a *part* of a design system, and each has a ceiling on what it can do on its own.

ARTEFACT	WHAT IT IS	LIVES IN	PRIMARY USERS	WHAT IT CANNOT DO ALONE
Style guide	Written rules for the brand and visual language: logo, colour, typography, imagery, voice.	PDF, wiki, brand site	Brand, marketing, designers	Guarantee that what ships matches. It describes; it doesn't build.
UI kit	Reusable visual elements in a design tool: styles, components, icons.	Figma (or similar)	Designers	Reach production. Engineers re-implement it by eye, so the kit and the product drift apart.
Component library	Coded, reusable UI components with an API (props, variants, states).	A code package (e.g. npm)	Engineers	Explain *when* and *why*. Without shared foundations it faithfully reproduces existing inconsistency.

ARTEFACT	WHAT IT IS	LIVES IN	PRIMARY USERS	WHAT IT CANNOT DO ALONE
Pattern library	Documented solutions to recurring problems: forms, empty states, filtering, onboarding.	Docs site	Designers, PMs, engineers	Enforce itself. Patterns are guidance that still has to be built from components.
Design system	All of the above, connected by tokens, with documentation, ownership and a process for change.	Design tool, code, docs, and a team	Everyone who builds the product	Succeed without adoption. It is only as real as the share of the product built with it.

CAPABILITY	STYLE GUIDE	UI KIT	COMPONENT LIBRARY	DESIGN SYSTEM
Documented visual decisions	●	◖	○	●
Tokens shared by design and code	○	◖	◖	●
Reusable design components	○	●	○	●
Reusable production code	○	○	●	●
Usage guidance (when / why)	◖	○	◖	●
Accessibility built in	○	◖	◖	●
Versioning and release process	○	◖	●	●
Contribution and governance	○	○	◖	●
Named owner(s)	◖	◖	◖	●

FIG 1.1 What each artefact typically covers. ● = core to the artefact, ◖ = sometimes, ○ = not covered. The design system column is not a bigger version of the others. It is the only one that connects design and code through shared tokens and owns the process for changing them.

COMMON MISTAKE

Calling a Figma library a design system

It is an easy and costly confusion. Leadership believes the consistency problem is solved; engineering, which never received anything it could import, keeps building its own versions. A UI kit is a legitimate and often correct first step (Chapter 02 explains when). Just call it what it is, so nobody budgets as if production is covered.

Which one do you have today?

Answer honestly. Each "no" marks the edge of what you currently have.

- [ ] Our colours, type sizes and spacing values exist as **named tokens**, not just hex codes in a file.
- [ ] Engineers **import** components from a shared package rather than rebuilding them per team.
- [ ] The design library and the code library use **the same names** for the same things.
- [ ] There is **written guidance** on when to use each component, not just what it looks like.
- [ ] Someone is **accountable** for the system, and it is in their job description.
- [ ] Changes are **versioned and announced**; teams can upgrade on purpose.
- [ ] Teams know **how to propose** a new component or change an existing one.

Seven yeses and you run a design system. One to three, you have a UI kit or component library with ambitions. That's fine, as long as it's a deliberate stage and not a stall.

01.3

The anatomy of a complete design system

A complete system has six layers that build on one another, and three capabilities that cut across all of them. Each layer only references the layer below it. That one-directional dependency is what lets a change at the bottom flow safely to the top.



FIG 1.2 The anatomy of a design system. Read bottom to top: each layer is built only from the layers beneath it. Documentation, tooling and governance span every layer. They are what turn a set of assets into a system people can trust.

LAYER	CONTAINS	WHO DECIDES	CHANGES HOW OFTEN
Principles	3–5 statements that settle trade-offs ("clarity over density", "accessible by default").	Design and product leadership	Rarely; years
Foundations	Colour, typography, spacing, grid, elevation, radius, motion, iconography. The subject of Part 02.	System team, with brand	Rarely; each change is high-impact
Tokens	Named, platform-agnostic values: primitive, semantic, component.	System team	Occasionally; via versioned releases
Components	Buttons, inputs, tables, dialogs, each with variants, states, API and a11y behaviour.	System team plus contributors	Continuously; minor releases

LAYER	CONTAINS	WHO DECIDES	CHANGES HOW OFTEN
Patterns	Compositions solving a user problem: search and filter, bulk actions, error recovery.	System team plus product teams	Continuously
Templates	Page scaffolds and layouts built from components and patterns.	Product teams, curated by system	As products evolve

A complete system does not mean a large one. A startup's complete system might be one page of principles, 120 tokens and 15 components, and still have all six layers and all three capabilities. Completeness is about connection, not volume.

01.4

How a design system connects the whole organisation

The most underrated function of a design system is linguistic. It gives design, engineering and product **one vocabulary for the interface.** When a designer says "primary button", an engineer types `<Button variant="primary">`, and both resolve to the token `color.action.primary`, a whole category of handoff error disappears.

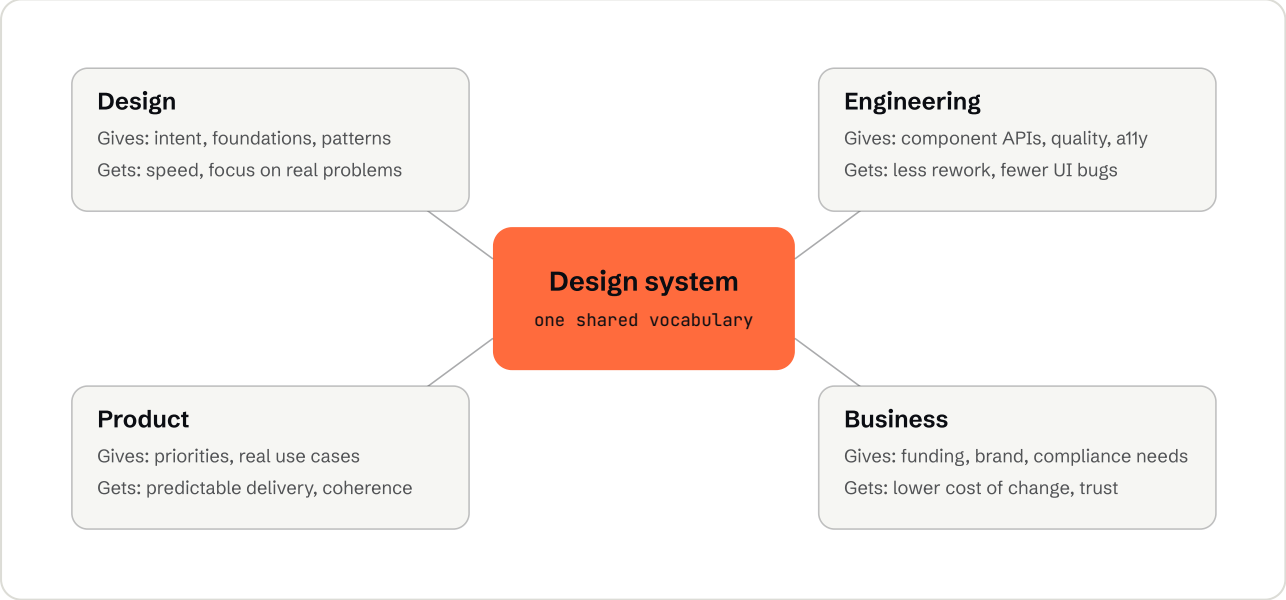


FIG 1.3 **The system as an exchange.** Every function contributes something and gets something back. A system that only takes (asks teams to adopt without giving them speed) or only gives (builds what nobody asked for) loses support quickly.

FUNCTION	WITHOUT A SYSTEM	WITH A WORKING SYSTEM
Design	Re-solves the same table, form and empty state on every project. Specs every pixel in handoff.	Assembles known parts and spends time on the problems that are actually new.
Engineering	Builds a "new" button per team; fixes the same focus bug in five places.	Imports tested components; fixes a bug once and ships the fix everywhere.

FUNCTION	WITHOUT A SYSTEM	WITH A WORKING SYSTEM
Product	Estimates swing because UI scope is unknown; design QA becomes a release gate.	UI scope is predictable; teams talk about behaviour, not padding.
Business	A rebrand is a multi-quarter programme; accessibility is a recurring audit finding.	Brand and accessibility changes are made at the token layer and released.

01.5

Why design systems matter more as products scale

A design system's value is not linear in company size. It compounds, for two reasons you can reason about without any vendor statistics.

Reason one: coordination grows faster than headcount

If every team that makes UI decisions has to stay consistent with every other team, the number of pairwise agreements grows as `n(n-1)/2`. That is the communication-path arithmetic Fred Brooks described for software teams.[5] Four teams need six agreements. Sixteen teams need 120. If instead each team agrees with one shared system, the number of agreements grows only as `n`.

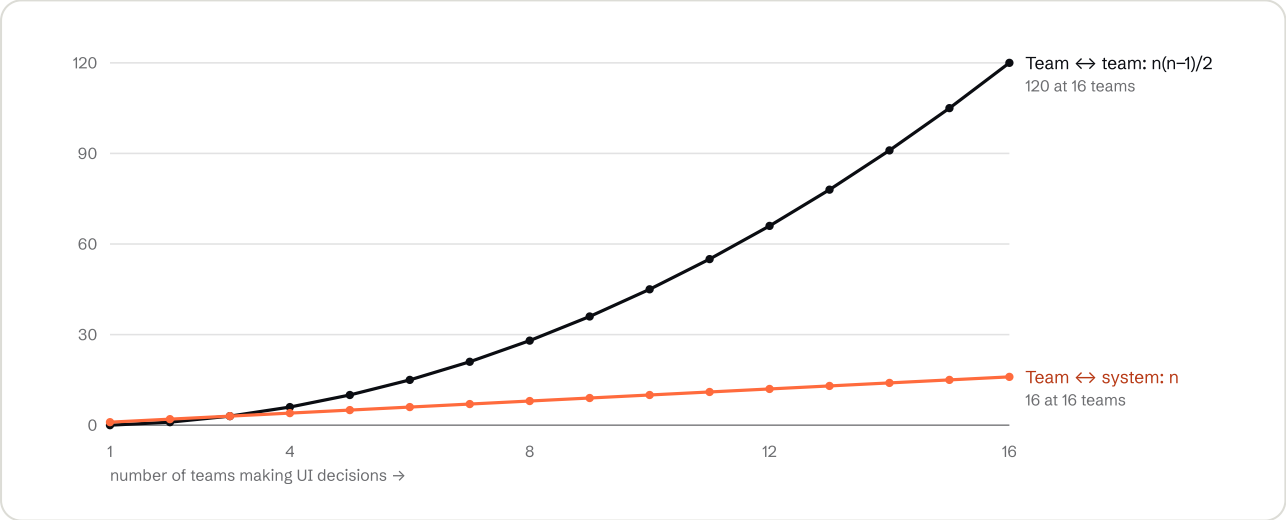


FIG 1.4 Agreements needed to stay consistent. ILLUSTRATIVE This is arithmetic, not survey data: it shows the shape of the coordination problem. No real organisation needs every pair of teams to talk, but the curve explains why consistency "suddenly" collapses somewhere between three and eight teams.

Reason two: the cost of change multiplies

Without tokens, changing a brand colour means finding every hard-coded value across every codebase, platform and design file: roughly *products × platforms × occurrences*. With a token architecture, it means changing one value and releasing. The same applies to an accessibility fix, a new dark theme or a typography refresh. The more surfaces you have, the more each "change once" saves. That's why the clearest returns usually appear at the second product, the second platform or the first rebrand, not at the first component.

This is the kind of problem design systems exist to solve. At CARS24, for example, the work Layer One's founder led put more than ten products in three countries on one colour and token system.[1] That outcome is only practical with a token layer between decisions and code.

PRACTICE LAYER ONE VIEW

Judge a design system by what it makes cheap

Good systems make the common case fast (building a standard screen) and the expensive case cheap (changing a decision everywhere). If yours does neither, it is a catalogue, not a system.

Glossary: the ten terms this playbook uses precisely

Foundation	A category of fundamental visual decision: colour, typography, spacing, layout, elevation, motion.
Design token	A named design decision stored in a platform-agnostic format and compiled into code (`space.400 = 16px`).
Primitive token	A raw value with a descriptive name and no usage intent (`color.blue.600`).
Semantic token	A token named for its purpose that points to a primitive (`color.text.link → color.blue.700`).
Component	A reusable piece of UI with a defined API, variants, states and accessibility behaviour.
Variant	A sanctioned alternative form of a component (primary, secondary, destructive).
Pattern	A reusable solution to a recurring user problem, composed of components.
Theme	A complete mapping of semantic tokens to values (light, dark, a brand).
Adoption	The share of the shipped product built with the system, measured in code, not in Figma.
Contribution	A change to the system proposed or built by someone outside the core team.

CHAPTER 02

Who needs a design system, and when

Every product already has a design system. It just might be implicit, inconsistent and stored in people's heads. The real question is how much of it to make explicit, and when.

READ THIS IF

You are deciding whether to invest now, later or not at all, or you are being asked to justify a system to someone who thinks it is premature.

WHAT YOU'LL LEAVE WITH

01 Direct answers to the seven questions founders and leaders ask most.

02 A five-stage maturity model to place your organisation, and to see the next stage.

03 A decision tree and readiness checklist you can use in a planning meeting.

02.1

The seven questions everyone asks

These answers are direct by design. Each one names the condition under which it would change.

Q

Does every company need a design system?

SHORT ANSWER — **Every company needs consistent design decisions. Not every company needs a dedicated, formal system.**

Any product with more than one screen has made decisions about colour, type, spacing and behaviour. The question is whether those decisions are written down, shared and reusable, or re-made by whoever builds the next screen. A two-person team with one product can stay consistent through conversation. Once decisions are made by people who don't talk daily, implicit systems start to fork. PRACTICE

Q

Is a design system only for enterprises?

SHORT ANSWER — **No. Small companies run them too, at a smaller scale.**

zeroheight's 2025 survey of design system practitioners found dedicated teams averaging about three people at companies with fewer than 100 employees, rising to about nine at companies with 500+.[6] DOCUMENTED (The sample is self-selected: people who answer a design systems survey tend to have one.) Enterprises need more *governance*, not more *system*. The core of a small system and a large one is the same: foundations, tokens and the components you use most.

Q

Does a startup need product-market fit first?

SHORT ANSWER — **Before product-market fit, build foundations, not a system.**

Pre-PMF, the product will change shape, so investing in a broad component library and governance is waste. But a token layer (named colours, a type scale, a spacing scale) costs days, not months, and it makes pivots *cheaper*: restyling a product built on tokens is a configuration change. Use an off-the-shelf component library (Radix, MUI and similar) themed with your own tokens, and defer custom components until the product stabilises. PRACTICE

Q

Is a team of 5–6 designers big enough?

SHORT ANSWER — **Usually yes. But count engineers and surfaces, not designers.**

Five designers on one product with eight engineers can stay coherent with a well-maintained UI kit and a shared token file. Five designers serving 40 engineers across a web app, a mobile app and an admin tool cannot. They will spend their week answering "which grey is it?" in Slack. The trigger for a real system is the number of *places UI gets built*, and how far apart they are. PRACTICE

Q

When is a UI kit enough instead of a full system?

SHORT ANSWER — **When the problem is design consistency, not delivery consistency.**

- One product, one platform, one codebase, and fewer than about three squads.
- Engineering already uses a themed third-party component library.
- Designers disagree with each other more than design disagrees with code.

A UI kit fixes design-to-design drift. It does nothing for design-to-code drift. When the second problem appears, the kit has reached its ceiling.

Q

How do we know we've outgrown what we have?

SHORT ANSWER — **When the cost of inconsistency shows up in delivery, not just in design reviews.**

The signals checklist in 02.4 is the practical test. If you tick five or more, the case for a system is already being paid for, just in rework instead of in investment.

Q

When is a design system unnecessary or premature?

SHORT ANSWER — **When nobody will own it, nobody will adopt it, or the product won't live long enough to benefit.**

- You are still searching for the product and expect to rebuild the UI within six months.
- One designer and one or two engineers build everything, and they sit together.
- The product is a marketing site or a campaign with a short life.
- Engineering leadership has not agreed to adopt it. A design-only system is a UI kit with a bigger budget.
- No one can be given protected time to own it. An unowned system decays faster than no system, because people stop trusting it.

02.2

A maturity model: where you are, and what's next

Design system maturity is a staircase, not a switch. Most organisations should aim for the *next* stage, not the last one. Skipping stages usually means building governance for components nobody uses yet.

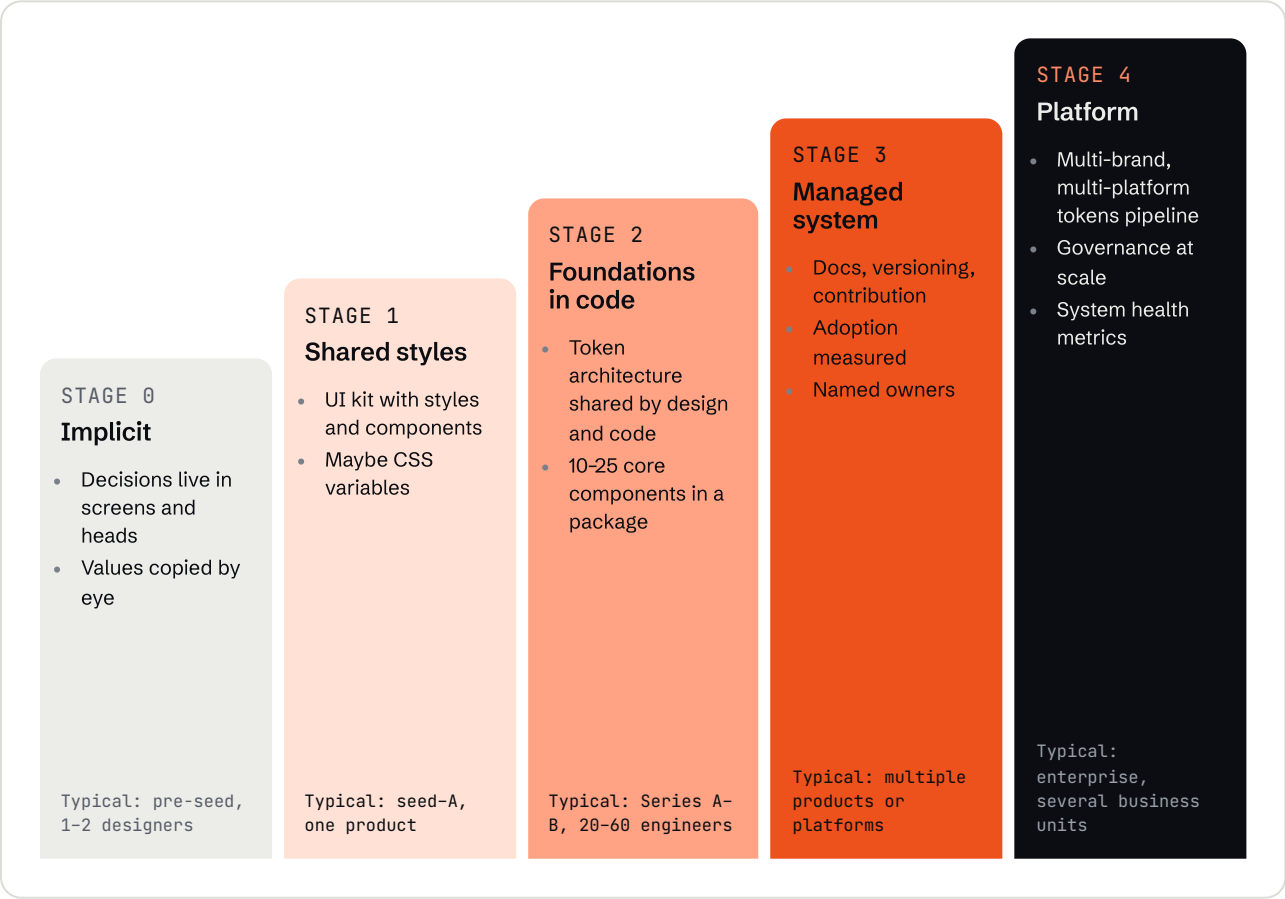


FIG 2.1 Five stages of design system maturity. PRACTICE The "typical" profiles are patterns from practice, not thresholds. A 30-person company with three white-label products may need Stage 3; a 300-person company with one product may be well served at Stage 2.

STAGE	YOU KNOW YOU'RE HERE WHEN…	THE MOVE TO THE NEXT STAGE	INVESTMENT TO GET THERE
0 Implicit	Designers ask each other for hex codes; engineers inspect screenshots.	Define foundations; publish a UI kit with styles.	1–3 weeks of one designer
1 Shared styles	Design is consistent; code still has many values that "look close".	Tokens as a single source of truth for design and code; build the most-used components.	1–3 months, a designer and an engineer, part-time
2 Foundations in code	New screens use the package; old ones don't; nobody owns requests.	Named owners, docs, release process, contribution path.	A small dedicated core (1.5–3 FTE)
3 Managed system	Multiple teams contribute; adoption is measured; releases are routine.	Theming, multi-platform output, governance across business units.	A dedicated team plus federated contributors
4 Platform	The system is infrastructure, with its own roadmap, SLAs and metrics.	Sustain; reduce cost of change; automate compliance.	Ongoing product-level budget

Ranges are PRACTICE estimates for orientation; Chapter 04 builds them up from roles and assumptions.

02.3

The decision tree

If you only have ten minutes in a leadership meeting, use this. It intentionally asks about *engineering* and *surfaces* before it asks about design.

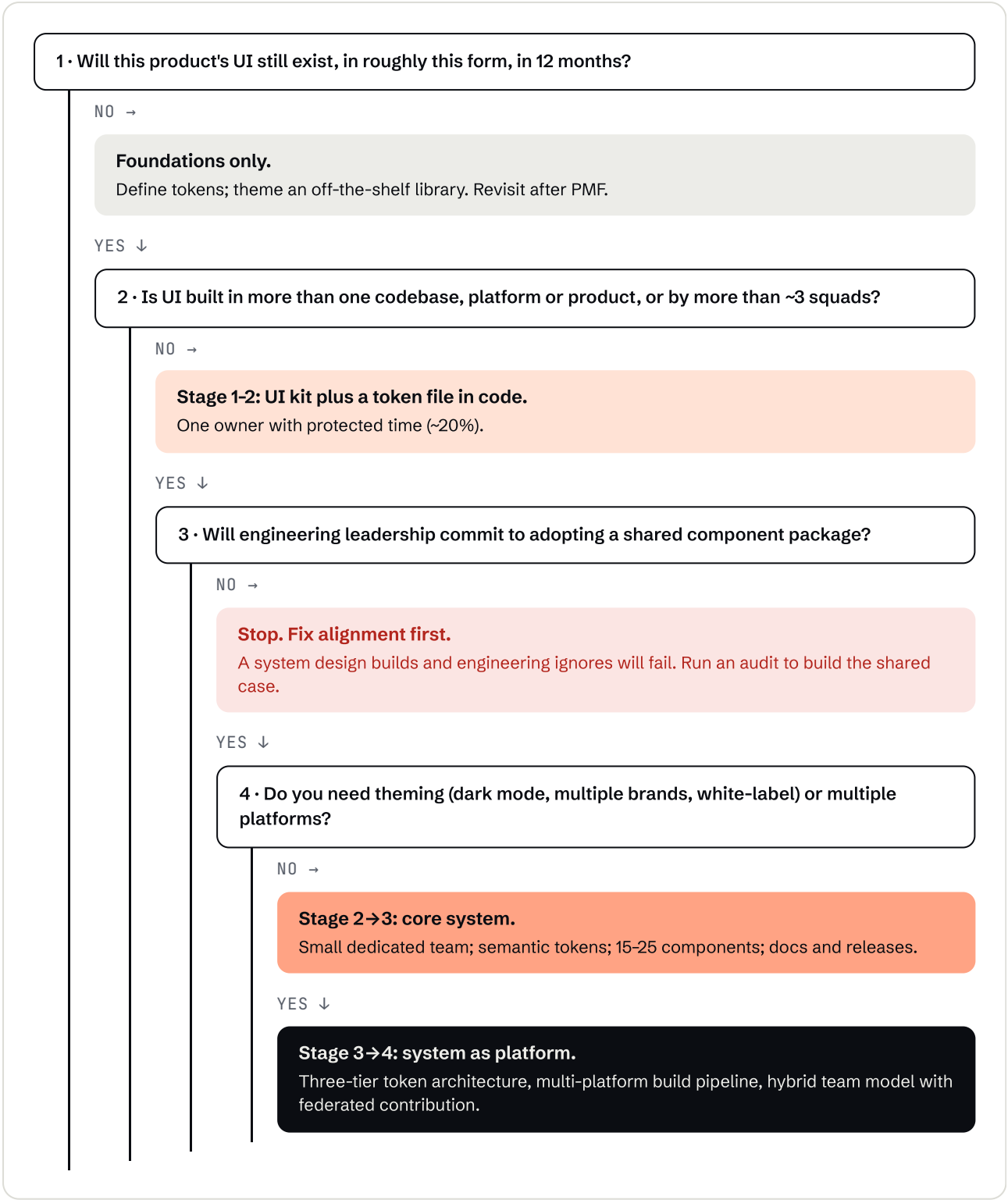


FIG 2.2 Should we invest, and how much? PRACTICE Question 3 is the one most often skipped, and the most common root cause of failed systems we see in audits.

02.4

Signs you have outgrown your current UI infrastructure

Each of these is observable, and most can be counted. Count them before you build the business case; they will become your baseline metrics in Chapter 03.

In the code

- [ ] A search for hard-coded hex values returns **hundreds of unique colours**.
- [ ] There is **more than one Button** (or Modal, or Table) component in the codebase.
- [ ] The same **accessibility bug** (focus, contrast, labels) has been fixed in several places.
- [ ] Engineers estimate a **brand colour change in weeks**, not hours.
- [ ] Front-end PR reviews regularly argue about **spacing and styling**.

In the organisation

- [ ] Design QA has become a **release gate** that blocks shipping.
- [ ] New designers or engineers take **weeks to learn** "how we do UI here".
- [ ] Customers or sales mention that products **"feel like different companies"**.
- [ ] A **second product, platform, brand or acquisition** is on the roadmap.
- [ ] Enterprise deals now include **accessibility conformance** requirements.

PRACTICE **HOW TO READ YOUR SCORE**

0–2: keep doing what you're doing; write down your foundations. **3–5:** you are paying for inconsistency in rework; plan Stage 2. **6+:** the system is already costing you more by its absence than it would by its presence. Start with an audit to size it.

02.5

Four scenarios

These are composite, fictional scenarios. They are not clients; they illustrate how the decision tree resolves in different contexts. ILLUSTRATIVE

SCENARIO A — SEED FINTECH

2 designers, 6 engineers, pre-PMF

Situation: one web app, pivoting monthly. The founder wants a "design system like Stripe's".

Recommendation: no system yet. Spend a week defining colour, type and spacing tokens; theme an open-source component library. **Why:** the product will change, and the tokens will survive the change while bespoke components won't.

SCENARIO B — SERIES B SAAS

7 designers, 45 engineers, 2 products

Situation: a web app plus a newly acquired analytics product with its own UI stack. Three button styles; dark mode requested by customers.

Recommendation: Stage 2→3. A two-week audit, then a semantic token layer that themes both products, and 15-20 shared components. Core team of ~2 FTE plus named contributors.

SCENARIO C — ENTERPRISE INTERNAL TOOLS

30+ tools built by different vendors

Situation: every tool looks different; accessibility is a compliance requirement; nobody owns UI centrally.

Recommendation: start with tokens and an accessibility baseline, not components. Tokens can be retro-fitted into legacy stacks; components can't. Establish a hybrid team model and a contribution path vendors must follow.

SCENARIO D — BRAND AND MARKETING SITE

Agency-built, redesigned every 2–3 years

Situation: the marketing team asks for a design system for the website.

Recommendation: a style guide plus a small set of CMS blocks is the right tool. Share colour and type *tokens* with the product so the brand stays aligned, but don't fund a product-grade system for a site with a short life.

02.6

Readiness checklist

Use this before committing budget. A "no" doesn't mean don't proceed. It means that item becomes the first workstream.

- ☐ **Sponsor.** A leader with budget authority who will defend the system in planning.
- ☐ **Engineering commitment.** Engineering leadership agrees teams will adopt the package.
- ☐ **Owner.** A named person with protected time; ownership is in the job description.
- ☐ **Audit.** You know what exists today: styles, components, tokens, a11y gaps.
- ☐ **Pilot team.** One product team volunteers to build with the system first.
- ☐ **Baseline metrics.** You've counted the signals in 02.4, so you can show change.
- ☐ **Scope.** You've chosen the next stage, not the final one.
- ☐ **Tech stack.** You know which frameworks and platforms the system must output to.

> ## Audit before you build.
>
> Rule #2 — Layer One

CHAPTER 03

What a design system is worth

"Consistency and efficiency" is not a business case. This chapter separates what has actually been measured from what is merely claimed, and gives you a way to measure your own outcomes.

READ THIS IF

You need to justify investment to a CFO, a board or a sceptical engineering leader, or you need to prove that an existing system is working.

WHAT YOU'LL LEAVE WITH

01 An honest picture of the published evidence, including its limits.

02 The mechanism behind each of nine business outcomes, so you can argue causally rather than by anecdote.

03 A KPI framework with formulas you can baseline this quarter.

03.1

Where the value actually comes from

Every business benefit of a design system traces back to three mechanisms. If a claimed benefit can't be traced to one of them, be suspicious of it.

MECHANISM 1

Decide once

Teams stop re-making the same decisions: which grey, how much padding, how errors behave. Time moves from re-deciding to the problems that are actually new.

MECHANISM 2

Build once

Components are built, tested and made accessible once, then reused. Defects are fixed once and the fix ships everywhere.

MECHANISM 3

Change once

Because products reference tokens and components rather than copies, a decision changed at the source propagates: rebrands, themes, accessibility fixes.

The first two produce savings that accrue steadily with usage. The third produces savings in lumps, at the moments of change, and it is often the larger of the three over a product's lifetime. Business cases that only count the first two undersell the system; business cases that promise the third without a token architecture overpromise it.

03.2

The evidence, honestly

The public evidence base for design system ROI is **thinner than the marketing around it suggests**. Many percentages in circulation trace back to a small number of studies, several of them run by vendors. That doesn't mean the benefits aren't real. It means you should rely on the mechanisms and measure your own outcomes, rather than borrowing someone else's number.

34%

faster completion of a design task when participants had a relevant, up-to-date design system.

DOCUMENTED Figma internal experiment.[7] Vendor-run; limited sample detail.

47%

faster to code a simple form with IBM's Carbon system than from scratch (median 2 h vs 4.2 h).

DOCUMENTED Sparkbox.[8] n = 8 developers; scratch build always done first.

79.1%

of the top million home pages had low-contrast text, the most common detectable WCAG failure.

DOCUMENTED WebAIM Million 2025.[9] A problem tokens can solve at source.

EVIDENCE LEDGER — WHAT IS KNOWN, AND HOW STRONGLY

FINDING	SOURCE AND METHOD	CAVEATS	HOW TO USE IT
Designers complete objectives ~34% faster with a system	Figma data science team; controlled task with and without a system; no-system group given older design files to reference.[7]	Vendor-published; the system used was directly relevant to the task, which is the best case.	As an upper bound for design-side efficiency on well-covered work.
Developers build a form ~47% faster with a system	Sparkbox; 8 developers each built the same form from scratch, then with Carbon.[8]	Tiny sample; order effect (second build benefits from familiarity with the design). Also showed better visual consistency.	As directional evidence, not a planning number.
Low contrast is the most common accessibility failure on the web	WebAIM automated scan of 1M home pages, 2025.[9]	Automated detection only; measures home pages, not apps.	To justify contrast-checked colour tokens (Chapter 07).
Most practitioner organisations now fund a dedicated system team	zeroheight Design Systems Report 2025; 294 respondents; 79% report a dedicated team.[6]	Self-selected sample of design-system practitioners.	As context for team-size benchmarks, not as proof of ROI.
Visual design shapes perceived usability and credibility	Kurosu & Kashimura (1995), the "aesthetic-usability" study;[10] Fogg et al. (2003), 2,500+ participants rating web credibility.[11]	Neither study is about design systems. They show that look and coherence affect trust; a system is one way to deliver coherence.	To explain why consistency is a revenue issue, not a cosmetic one.

COMMON MISTAKE

Putting someone else's percentage in your business case

"Design systems make teams 47% faster" will be challenged by the first engineer who reads the study, and your credibility goes with it. Use published findings to show the mechanism is real, then build your case on your own baseline using the KPIs in 03.4. Chapter 04 shows how to do this with conservative assumptions.

03.3

Nine outcomes, and the mechanism behind each

For each outcome below, the *mechanism* column is the argument you make to leadership; the *measure* column is how you prove it later.

OUTCOME	MECHANISM	WHAT TO MEASURE	EVIDENCE
Design and delivery velocity	Decide once: designers assemble known parts and spec less in handoff.	Time from brief to dev-ready design for comparable features; handoff comment count.	DOCUMENTED
Engineering efficiency	Build once: no re-implementing standard UI; less styling code to review.	Cycle time for UI-heavy tickets; lines of custom CSS per feature; PR review rounds on styling.	DOCUMENTED
Reuse, less duplication	One implementation per component instead of one per team.	Number of distinct implementations of the same component across repos; % of UI imported from the package.	PRACTICE
Quality and consistency	Tested components with defined states; tokens remove "close enough" values.	UI defects per release; unique hard-coded colour and spacing values over time.	DOCUMENTED
Accessibility and compliance	Contrast, focus, target size and semantics are solved in the system and inherited.	Automated a11y violations per page; audit findings per release; time to remediate.	STANDARD
Maintenance and scalability	Change once: a fix or update in the system reaches every consumer through a version bump.	Effort (hours) for a cross-product visual change; time to ship a new theme.	PRACTICE
Cross-team collaboration	A shared vocabulary removes translation between design and code.	Survey: "I know which component to use"; design–eng clarification threads per feature.	PRACTICE
Time to market	New products and features start from a working kit instead of a blank page.	Time from project start to first usable prototype or beta for new products.	PRACTICE
Product trust and perceived quality	Coherent, polished interfaces are judged more usable and credible.	Perceived-quality questions in product surveys; support tickets citing confusion; sales feedback.	DOCUMENTED

Evidence column: DOCUMENTED where published studies support the mechanism (03.2); STANDARD where a standard defines the requirement (WCAG 2.2[12]); PRACTICE where the case rests on practice and your own measurement.

Accessibility deserves its own line in the budget

Accessibility is the outcome where the business case has moved fastest. WCAG 2.2 sets testable success criteria for contrast, focus visibility and target size.[12] STANDARD Regulation increasingly references these criteria: the European Accessibility Act, for example, applies to many digital products and services placed on the EU market from 28 June 2025.[13] A design system is the most efficient place to meet them, because a contrast-checked colour token or a component with correct focus behaviour fixes the issue in every product that uses it. Remediating screen by screen fixes it once, and only until the next screen is built.

03.4

A KPI framework you can baseline this quarter

> ### Measure time saved, not usage.
>
> Rule #8 — Layer One

Usage metrics (component import counts, Figma insertions) tell you whether the system is *used*. They don't tell you whether it is *worth it*. Track both, but report outcomes to leadership.

KPI	FORMULA	HOW TO COLLECT	CADENCE
Adoption	`UI files importing the system ÷ all UI files`	Static analysis of imports across repos	Monthly
Token coverage	`1 - (hard-coded colour + spacing values ÷ total style values)`	Lint rule or a scripted search for raw values	Monthly
Duplication	`Distinct implementations of component X`	Code search for known component patterns	Quarterly
UI cycle time	`Median(ticket start → merge) for UI tickets, system vs non-system`	Issue tracker plus PR data, tagged	Quarterly
Design QA defects	`Visual + a11y defects found after dev ÷ features shipped`	Bug tracker labels	Per release
Accessibility	`Automated violations per page (axe or similar)`	CI accessibility checks on key flows	Per release
Cost of change	`Hours to ship a cross-product visual change`	Time-box and log real changes (e.g. a colour update)	Per event
Contribution health	`Median days from proposal to release; proposals per quarter`	Contribution tracker	Quarterly

KPI	FORMULA	HOW TO COLLECT	CADENCE
Satisfaction	`% agree: "The system makes my work faster"`	Short survey of designers and engineers	Twice a year

PRACTICE BASELINING TIP

Measure before you build

Run the duplication, token-coverage and a11y counts during the audit, before any system work ships. Without a baseline, every later number is an opinion. With one, a single slide ("unique colour values: 412 → 61") does more for continued funding than any industry statistic.

03.5

When value shows up, and when it doesn't

MONTHS 0–3

Consistency of new work

Foundations and first components land. New screens look coherent. Savings are small; the system is still a cost.

MONTHS 3–12

Build-once savings

Adoption passes the tipping point; teams stop rebuilding standard UI; QA defects fall. Value begins to approach cost.

MONTHS 12+

Change-once savings

Themes, rebrands, new products and accessibility fixes become configuration. This is where most systems pay back.

FIG 3.1 The value curve of a design system. PRACTICE Timings vary with team size and adoption speed. The shape is consistent: cost first, steady savings next, large savings at moments of change.

When a design system destroys value

- **Nobody adopts it.** A system with 10% adoption has all of the cost and almost none of the benefit. Adoption is the multiplier on every outcome above.
- **It is too abstract.** Components with 40 props that try to serve every case are slower to use than writing the UI by hand.
- **It is a bottleneck.** If every new need waits weeks for the system team, product teams will route around it, and they will be right to.
- **It is frozen.** A system that doesn't evolve with the product becomes a constraint teams fight rather than a tool they use.

Chapter 05 is about the operating model that prevents all four.

CHAPTER 04

The economics of design systems

The biggest misconception about design systems is that they need a large upfront investment. They don't. Investment is a dial, not a switch. This chapter shows how to set the dial and how to know when to turn it up.

READ THIS IF

You need a budget number, a staffing plan or an ROI model, or someone has quoted you a seven-figure estimate and you want to understand what drives it.

WHAT YOU'LL LEAVE WITH

01 The ten factors that drive cost, so you can scope rather than guess.

02 Three budget scenarios built from roles and FTE-months, with every assumption stated.

03 An ROI model you can run with your own numbers, plus an honest view of when it breaks even.

04.1

How to read the numbers in this chapter

Salaries differ by a factor of five or more between markets, so a dollar figure from one market is meaningless in another. This chapter therefore budgets in **FTE-months** (one full-time person for one month) and converts to currency only at the end, using three example rates you should replace with your own.

ILLUSTRATIVE ALL FIGURES IN THIS CHAPTER ARE ILLUSTRATIVE

They are built from stated assumptions, not measured from real projects. The point is the *method*: change the assumptions to match your organisation and the model recalculates. The full model is in `scripts/generate-data.mjs` in the playbook's repository.

04.2

What determines the investment

Ten factors explain most of the difference between a system that costs a few FTE-months and one that costs a hundred.

COST DRIVER	LOWER COST WHEN…	HIGHER COST WHEN…	IMPACT
Platforms	Web only	Web + iOS + Android (+ desktop): each needs native components	**Very high**
Frameworks	One front-end framework	React and Angular and a legacy stack in parallel	**Very high**
Brands and themes	One brand, light mode	Dark mode, multiple brands, white-label for customers	**High**
Existing debt	Young product, little UI code	Years of divergent UI to migrate	**High**
Component scope	15–25 core components	60+ components including complex data tables, charts, editors	**High**
Accessibility target	WCAG 2.2 AA built in from the start	Retrofitting AA (or AAA) onto existing components	**Medium**
Documentation depth	Usage guidance and code examples	Full public docs site, content guidelines, tutorials	**Medium**
Team distribution	Co-located, one time zone	Many teams, vendors, time zones	**Medium**
Tooling and automation	Manual token export, simple package	Token pipeline, visual regression tests, linting, Figma-code sync	**Medium** (pays back)
Governance overhead	One product team as owner	Councils, multiple business units, compliance sign-off	**Medium**

Platforms and frameworks dominate. Every additional platform roughly means another implementation of every component. That's why token architecture comes first: tokens are the one layer that is written once and compiled to every platform.

04.3

Roles and resources

ROLE	RESPONSIBLE FOR	WHEN YOU NEED IT
System lead / product owner	Roadmap, prioritisation, stakeholder alignment, adoption, metrics. Treats the system as a product.	From Stage 2; can be 0.5 FTE of a design or engineering lead
System designer	Foundations, component design, states, documentation of usage, Figma libraries.	From Stage 1
System engineer (front end)	Token pipeline, component code and API, testing, release and package management.	From Stage 1. The role most often missing.
Native engineers	iOS and Android component implementations from the same tokens.	When you ship native apps
Accessibility specialist	Standards, audits, screen-reader testing, review of component behaviour.	Part-time from Stage 2
Content / docs	Docs site structure, writing guidelines, UX writing patterns.	Part-time from Stage 3

ROLE	RESPONSIBLE FOR	WHEN YOU NEED IT
Contributors	Product designers and engineers who propose and build changes.	From Stage 3; time protected in their teams' plans

COMMON MISTAKE

Staffing a design system with designers only

It produces an excellent Figma library and no adoption, because adoption happens in code. Our rule of thumb is at least as much engineering capacity as design capacity on the core team, and more once you ship multiple platforms. PRACTICE

04.4

How long it takes

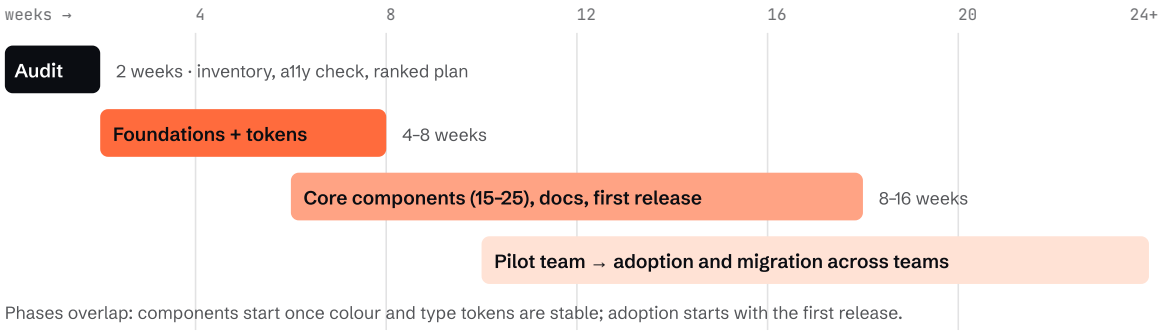


FIG 4.1 A typical first-system timeline for a single-platform product. PRACTICE Roughly four to six months to a usable first release, then adoption becomes the main work. Multiply component effort per additional platform.

04.5

Three budget scenarios

Each scenario lists year-one effort by role, then converts the total at three example loaded monthly costs per person (salary plus overheads): $6,000, $12,000 and $20,000. These are example inputs, not market data; substitute your own. ILLUSTRATIVE

YEAR-ONE INVESTMENT

SCENARIO	PROFILE	BUILD FTE-MO	RUN FTE-MO	TOTAL	@ $6,000/MO	@ $12,000/MO	@ $20,000/MO
Startup Stage 1 → 2	1 product, web, 2–3 designers, 8–15 engineers	3.0	3.6	**6.6**	$39,600	$79,200	$132,000
Mid-sized SaaS Stage 2 → 3	2–3 products, web + mobile web, 6–12 designers, 40–80 engineers	22.2	22.8	**45.0**	$270,000	$540,000	$900,000
Enterprise Stage 3 → 4	5+ products, web + iOS + Android, multiple brands, 30+ designers, 200+ engineers	108.0	0.0	**108.0**	$648,000	$1,296,000	$2,160,000

HOW EACH TOTAL IS BUILT UP

SCENARIO	PHASE	ROLE	ALLOCATION	FTE-MONTHS
Startup	Build	Design system designer	0.5 FTE × 3 mo	1.5
		Frontend engineer	0.5 FTE × 3 mo	1.5
	Run	Designer (embedded owner)	0.2 FTE × 9 mo	1.8
		Engineer (embedded owner)	0.2 FTE × 9 mo	1.8
Mid-sized SaaS	Build	Design system lead (0.5)	0.5 FTE × 6 mo	3.0
		Design system designer	1 FTE × 6 mo	6.0
		Design system engineers	2 FTE × 6 mo	12.0
		Accessibility review	0.2 FTE × 6 mo	1.2
	Run	Design system lead (0.5)	0.5 FTE × 6 mo	3.0
		Design system designer	1 FTE × 6 mo	6.0
		Design system engineers	1.5 FTE × 6 mo	9.0
		Product-team contributions	0.8 FTE × 6 mo	4.8
Enterprise	Build	Design system lead / PM	1 FTE × 12 mo	12.0
		Design system designers	2 FTE × 12 mo	24.0
		Web engineers	3 FTE × 12 mo	36.0
		iOS + Android engineers	2 FTE × 12 mo	24.0
		Accessibility specialist	0.5 FTE × 12 mo	6.0
		Docs / content	0.5 FTE × 12 mo	6.0

FIG 4.2 Year-one investment by scenario, in FTE-months. ILLUSTRATIVE The startup scenario uses part-time embedded owners, not hires. The enterprise scenario has no separate run phase because a year-one enterprise system is still being built. Not included: the effort product teams spend migrating to the system, which is usually larger than the system team's own cost and should be planned with each team.

Reading the scenarios

- **The startup system costs less than one engineer for a year.** It buys tokens, a themed off-the-shelf library and a handful of custom components: enough to stay coherent through Series A.

- **The mid-sized system is a small team, not a department.** Two to three people are enough if the scope is one platform plus mobile web and product teams contribute.
- **Enterprise cost is driven by platforms and brands.** Engineering is over half of the enterprise effort here, and native iOS and Android work alone is more than a fifth of it.

04.6

Dedicated team or existing product team?

	EMBEDDED OWNERSHIP (PRODUCT TEAM OWNS IT)	DEDICATED CORE TEAM
How it works	One designer and one engineer get protected time (often 20–30%) to own the system within their product team.	A team whose product *is* the system, serving product teams as customers.
Strengths	Cheap; close to real needs; no "ivory tower".	Focus; consistent quality; can serve many teams and platforms.
Risks	Protected time gets eaten by product deadlines; system bends toward one product.	Distance from product reality; becomes a bottleneck if it doesn't enable contribution.
Fits	One product, fewer than ~30 engineers, Stage 1–2.	Two or more products or platforms, ~40+ engineers, Stage 3+.

For comparison, zeroheight's 2025 survey reports average dedicated team sizes of about three people at companies under 100 employees, four at 100–499, and nine at 500+.[6] DOCUMENTED Teams rarely grow beyond 20–25 people, even in very large organisations, because at that scale contribution from product teams does the scaling.

04.7

Ongoing maintenance costs

The most expensive mistake in design system budgeting is funding the build and not the run. A system without maintenance degrades within a year: components lag behind product needs, teams fork them, and adoption reverses.

What "run" includes

- [ ] Bug fixes and accessibility fixes in components
- [ ] New variants and components as products evolve
- [ ] Dependency and framework upgrades (e.g. a React major version)
- [ ] Design tool library maintenance and releases
- [ ] Documentation, office hours, onboarding
- [ ] Reviewing and merging contributions
- [ ] Measuring adoption and health

PRACTICE **BUDGET RUN AS A TEAM, NOT A PERCENTAGE**

Rather than "maintenance = X% of build", plan for the team that built the system to stay, at a similar or slightly smaller size, with its time shifting from building to enabling. In our experience, systems whose builders are reassigned after launch rarely keep adoption.

04.8

Calculating ROI

A defensible ROI model has three parts: hours saved by reuse, avoided costs at moments of change, and the cost of the team. The formulas:

```
// Annual value from reuse (efficiency)
hours_saved = people × focused_hours × UI_share × adoption × efficiency_gain
value       = hours_saved × loaded_hourly_rate

// Annual value from change-once events (count only those actually planned)
avoided     = Σ (cost_without_system - cost_with_system) for rebrands, themes, a11y fixes

// Return
ROI         = (value + avoided - system_cost) ÷ system_cost
```

A worked example: Ledgerly

Ledgerly, our fictional B2B finance product, has 40 engineers and 6 designers. We assume a 20% efficiency gain on work the system covers. That is deliberately well below the 34% and 47% figures from 03.2, which come from best-case tasks. Adoption ramps over three years. We count *only* efficiency savings, so the model leaves out every change-once event.

YEAR	ADOPTION	HOURS SAVED	VALUE	TEAM COST	NET	ROI
Year 1 Build year: foundations + core components, adoption ramps	40%	1,664 h	$149,760	$288,000	-$138,240	-48%
Year 2 Most new work uses the system	70%	2,912 h	$262,080	$288,000	-$25,920	-9%
Year 3 Migration of older screens largely complete	85%	3,536 h	$318,240	$288,000	$30,240	11%

Assumptions: 40 engineers, 6 designers · 1600 focused h/FTE-year · 25% of engineering and 50% of design time is UI production · efficiency gain on system-covered work 20% · team of 2 FTE at $12,000/FTE-month ($90/h). Excludes change-once events, accessibility remediation and quality effects.

FIG 4.3 Ledgerly: a 2-person system team, efficiency savings only. ILLUSTRATIVE On reuse alone, a two-person team at this company size becomes net-positive in year three, and recovers its earlier deficit only after that. Any single change-once event, such as a dark theme, a rebrand or an accessibility remediation programme, typically moves break-even earlier.

That result is deliberately unflattering, and it is the most useful number in this chapter. It shows that at 40 engineers, efficiency alone barely justifies a two-person dedicated team. The case rests on adoption speed and on change-once events, which is exactly where to focus the plan.

LEDGERLY — ANNUAL NET VALUE VS A 2-FTE TEAM ($288,000/YR)

EFFICIENCY GAIN ↓ · ADOPTION →	40%	70%	90%
10%	-$213,120	-$156,960	-$119,520
20%	-$138,240	-$25,920	$48,960
34% (Figma figure)	-$33,408	$157,536	$284,832

FIG 4.4 How sensitive is the answer? ILLUSTRATIVE Adoption matters as much as efficiency. A system at 90% adoption with a modest 10% gain beats one at 40% adoption with a 34% gain. That is why Chapter 05 treats adoption as the system team's primary job.

ENGINEERS NEEDED FOR EFFICIENCY SAVINGS ALONE TO COVER THE TEAM (70% ADOPTION, 20% GAIN)



Sensitivity: 2 FTE at 70% adoption and 34% gain → 27 engineers · 2 FTE at 50% adoption and 20% gain → 63 engineers.

FIG 4.5 Break-even team size. ILLUSTRATIVE With these assumptions, each full-time system person needs roughly 22–23 engineers' worth of UI work to cover their cost from efficiency alone. Use this to sanity-check proposals: an 8-person system team for 60 engineers needs a strong change-once case.

04.9

Start small, invest progressively

Fund each stage from the evidence produced by the last one. Each step has a *gate*: a measurable condition that unlocks the next round of investment.

01 **Audit — 2 weeks**

Inventory styles, components and tokens in design and code; baseline duplication, token coverage and accessibility. **Gate:** leadership agrees on the ranked problem list.

02 **Foundations and tokens — 1–2 months, 1–2 people part-time**

Colour, type, spacing, layout and elevation tokens in code and design. **Gate:** one product team ships new work using only tokens.

03 **The ten most-used components — 2–3 months**

Pick by frequency in the audit, not by what's interesting. **Gate:** pilot team measurably faster or fewer UI defects.

04 **Docs, releases and contribution — ongoing**

Make it usable by teams who didn't build it. **Gate:** a second team adopts without hand-holding; first external contribution merged.

05 **Scale — themes, platforms, governance**

Only now invest in multi-brand, native platforms and formal governance. **Gate:** adoption above ~60% and a planned change-once event that needs it.

THE ECONOMICS IN ONE SENTENCE

Tokens are cheap and pay back immediately; components pay back with adoption; governance pays back with scale. Fund them in that order.

CHAPTER 05

Design system operating models

A design system is not something a team builds and publishes. It is a product with users, a roadmap, releases, support and metrics. The operating model decides whether it is still trusted in three years.

READ THIS IF

You are setting up a system team, your system has adoption problems, or product teams see the system team as a bottleneck.

WHAT YOU'LL LEAVE WITH

01 A clear choice between centralised, federated and hybrid models, and what each requires.

02 Working templates: decision rights (RACI), contribution flow, versioning policy.

03 A health scorecard and a plan for managing design and technical debt.

05.1

Treat the system as a product

The most reliable predictor of a healthy design system is whether its team behaves like a product team. That means it knows its users (product designers and engineers), runs discovery on their problems, prioritises a roadmap, ships on a cadence, supports what it ships and measures outcomes.

PRODUCT PRACTICE	IN A DESIGN SYSTEM TEAM
Users	Product designers, engineers, PMs, content designers, and sometimes vendors and partners.
Discovery	Audit requests, sit with product teams, review what teams build *outside* the system and why.
Roadmap	Public, prioritised by reach (how many teams) × pain, not by what the system team finds interesting.
Releases	Versioned, with changelogs and migration notes; on a predictable cadence.
Support	A channel with response targets, office hours, onboarding sessions.
Metrics	Adoption, coverage, contribution, satisfaction and outcome KPIs (03.4).

05.2

Centralised, federated and hybrid models

Nathan Curtis's 2015 essay set out the vocabulary most teams still use: *solitary*, *centralised* and *federated*.[14] Salesforce later described a *cyclical* hybrid, with a central team plus a federated group of contributors that inform each other.[15] Curtis himself later concluded that successful systems always have a central team *and* seek participation from a federated community.[16] DOCUMENTED

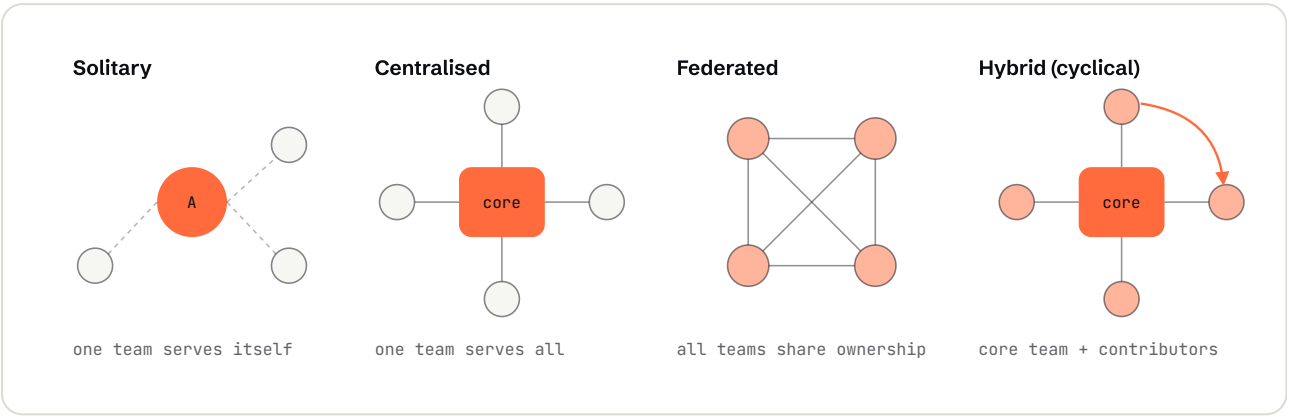


FIG 5.1 **Four team models.** Orange marks system ownership; tinted nodes are product teams that contribute. In the hybrid model, contributions flow from product teams into the core and releases flow back out.

MODEL	STRENGTHS	FAILURE MODE	FITS
Solitary	Fast; grounded in real product needs.	The system bends to one product; others don't adopt.	Stage 1–2; a single product
Centralised	Focus, consistent quality, clear accountability.	Becomes a bottleneck and drifts from product reality ("overlords don't scale").	Early Stage 3, while foundations stabilise
Federated	Spreads load; strong product context.	Nobody's job, so nobody does it; decisions stall.	Rarely works alone
Hybrid	Core quality plus contribution at scale.	Needs real investment in contribution process and reviewer time.	Stage 3–4; multiple products

PRACTICE **OUR RECOMMENDATION**

Start solitary or centralised to establish foundations and the first components. Move to hybrid as soon as two or more teams are adopting. Do it deliberately, by budgeting contributor time in product teams' plans, not by hoping for goodwill.

05.3

Ownership and decision rights

Most system conflicts are really unclear decision rights. Write them down. The RACI below is a starting template for a hybrid model. **R**esponsible does the work, **A**ccountable makes the call, **C**onsulted gives input, **I**nformed is told.

DECISION	SYSTEM LEAD	SYSTEM DESIGNERS	SYSTEM ENGINEERS	PRODUCT TEAMS	DESIGN / ENG LEADERSHIP
Change a foundation (palette, type scale)	**A**	R	C	C	C
Add a new component to the core	**A**	R	R	C	I
Add a variant to an existing component	I	**A**	R	R	—
Component API and breaking changes	C	C	**A**/R	C	I
Accessibility conformance of components	**A**	R	R	I	I
Use of a local (non-system) component	I	C	C	**A**/R	—
Deprecate and remove a component	**A**	R	R	C	I
System roadmap and funding	R	C	C	C	**A**

Note row six. Product teams are *accountable* for their own local components. A healthy system lets teams build outside it when it doesn't meet their need. It just asks them to say so, so the system can learn.

05.4

Design and engineering collaboration

Design-engineering friction in a system usually comes from two sources of truth disagreeing. Resolve it by deciding, explicitly, what each source is authoritative for.

TOKENS

Truth for values

One token source (JSON in the repository, in the DTCG format) feeds both the design tool and code.[17] Neither side hand-edits values.

CODE

Truth for behaviour

States, keyboard interaction, responsiveness and accessibility semantics are defined by the shipped component.

DESIGN LIBRARY

Truth for intent

Usage, composition and visual specification. It mirrors code names and variants one-to-one.

One definition of done for a component

- [ ] Designed for every state: default, hover, focus, active, disabled, loading, error, empty.
- [ ] Built using tokens only. No raw values in component code.
- [ ] Keyboard and screen-reader behaviour specified and tested; meets WCAG 2.2 AA.[12]
- [ ] Names, variants and properties identical in the design library and in code.
- [ ] Documented: when to use, when not to, anatomy, examples, accessibility notes.
- [ ] Released in a versioned package with a changelog entry.

05.5

Contribution and review

Contribution is what lets a small core team serve a large organisation. It only works if the path is clear, proportional to the size of the change, and fast.

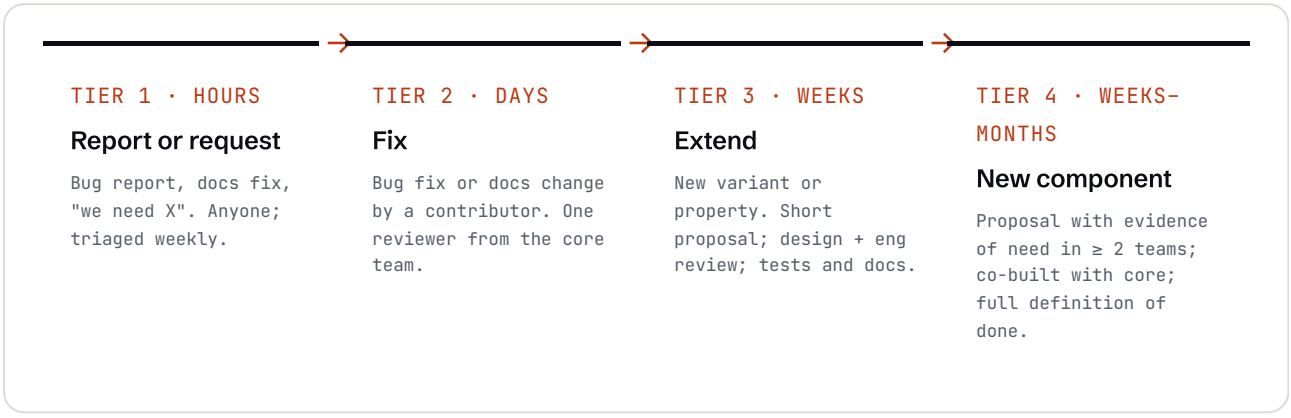


FIG 5.2 Four tiers of contribution. PRACTICE Review effort scales with the change. The most common failure is applying Tier 4 process to a Tier 2 fix, which teaches contributors not to bother.

The rule of two

A component enters the core system when **at least two teams** need it. Until then, it lives as a local component in the team that built it, built from system tokens so it can be promoted later. This prevents the core from filling with one-off components, and it gives the system a natural pipeline of tested candidates. PRACTICE

05.6

Governance and decision-making

Governance is how a system decides when people disagree. Keep it lightweight and written down:

01 **Principles settle most arguments**

"Accessible by default" and "clarity over density" end debates faster than any committee.

02 **RFCs for significant changes**

A one-page proposal (problem, options, recommendation, migration impact) open for comment for a fixed period, decided by the accountable owner.

03 **A small council for cross-org decisions**

At Stage 4, representatives of major product areas meet on a fixed cadence for decisions that affect everyone: foundations, breaking changes, roadmap.

04 **A documented exception path**

Teams may diverge with a stated reason. Exceptions are logged and reviewed quarterly; recurring exceptions become roadmap items.

> **Every decision comes with a reason.**
>
> Rule #6 — Layer One

05.7

Versioning and release management

Product teams need to upgrade on purpose, not discover changes in production. Use semantic versioning:[18] `MAJOR.MINOR.PATCH`, and define what each means *for a design system*. Visual changes are the grey area most policies forget.

BUMP	MEANING	DESIGN SYSTEM EXAMPLES
MAJOR	Consumers must change their code or will see breaking behaviour.	Renamed or removed token; changed component API; removed variant; changed default size or spacing that alters layouts.
MINOR	New capability, backward compatible.	New component, variant, token or icon; new optional property.
PATCH	Fixes, no intended visual or API change.	Bug fix; accessibility fix that doesn't change layout; documentation.

Release practice

- A predictable cadence (e.g. minor releases every two weeks), with patches as needed.
- Changelog entries written for consumers ("what changes for you"), not for the system team.
- Design library versions published in step with code releases.
- Visual regression tests on every component before release.

Deprecation policy

- Deprecate in a minor release, with a console warning and docs notice.
- Provide a migration guide, and a codemod where the change is mechanical.
- Remove only in the next major release, after a stated minimum period.
- Track usage of deprecated items so you know when removal is safe.

05.8

Documentation and adoption

> **Docs decide adoption.**
>
> Rule #9 — Layer One

Engineers and designers adopt what they can understand in the five minutes before a deadline. Every component page should answer, in order:

01 **What is it, and when do I use it?**
One sentence, plus when *not* to use it and what to use instead.

04 **What do I need to know about accessibility?**
Keyboard behaviour, labelling requirements, known pitfalls.

02 **What does it look like?**
Live examples of variants and states.

03 **How do I build with it?**
Copyable code; properties table; design library link.

05 **Do's and don'ts**
Visual examples of correct and incorrect usage.

06 **What changed?**
Version history and migration notes.

Adoption tactics that work

Pilot, then case study
Ship with one team first; publish their before-and-after internally.

Champions
One named designer and engineer per product team who get early access and a direct line.

Office hours
A weekly slot where anyone can bring a screen and get help building it with the system.

Lint rules
Flag raw colour values and deprecated components in code review, automatically.

Codemods
Automate mechanical migrations so upgrades cost minutes, not sprints.

Make the right way easiest
Starter templates and scaffolds that begin with the system already wired in.

05.9

Measuring system health

Review a small scorecard monthly with the team and quarterly with leadership. Look for trends, not targets.

DIMENSION	METRIC	HEALTHY SIGNAL	WARNING SIGNAL
Adoption	% of UI files importing the system; % of products on a supported version	Rising; most products within one major version	Flat; products stuck on old majors
Coverage	Token coverage; local components per product	Hard-coded values falling	Local components growing faster than core
Quality	Open bugs; a11y violations; visual regressions	Bugs closed faster than opened	Same bug class recurring
Flow	Median time from request to release; contribution count	Days to weeks; contributions growing	Months; contributions falling to zero
Sentiment	"The system makes my work faster" (% agree)	Majority agree and rising	Teams describe the system as a blocker

05.10

Managing technical and design debt

Every system accumulates debt: components that no longer fit, workarounds that became permanent, values that drifted. Debt is normal; *invisible* debt is the problem.

DEBT TYPE	HOW IT APPEARS	HOW TO DETECT IT
Detached instances	Designers break components to make one-off changes.	Design-tool analytics on detachments by component
Hard-coded values	Raw colours, sizes and spacing in product code.	Lint rules; scripted search
Forked components	Teams copy a system component and modify it.	Code search for copied component signatures
Deprecated usage	Old components still in production after replacement.	Import tracking of deprecated items
Inaccessible patterns	Components that pass in isolation but fail in composition.	Automated checks on key flows; periodic manual audits
Design-code drift	The design library and code no longer match.	Periodic side-by-side review; token sync checks

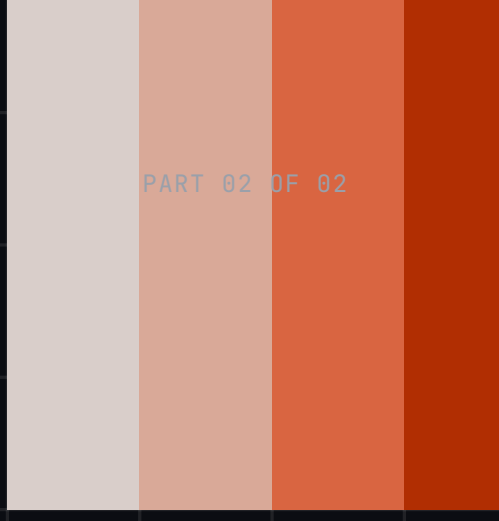
PRACTICE **A DEBT BUDGET**

Keep a visible debt register, and reserve a fixed share of every cycle (we suggest around a fifth) for paying it down. Teams that schedule debt work on a fixed basis rarely need a "big rewrite"; teams that don't usually end up doing one.

Operating model checklist

- [ ] Team model chosen and written down
- [ ] RACI for the eight decisions in 05.3
- [ ] Tokens, code and design library each own a defined truth
- [ ] Definition of done for components
- [ ] Contribution tiers and the rule of two published
- [ ] Versioning policy including visual changes
- [ ] Deprecation policy with minimum notice period
- [ ] Docs template for every component
- [ ] Health scorecard with a monthly review
- [ ] Debt register and debt budget

> **Leave the team able to run it without us.**
>
> Rule #10 — Layer One



02

Building the foundations of a design system

Colour, typography, spacing and layout are the decisions every component inherits. Get them right and the rest of the system gets easier; get them wrong and every component carries the mistake. This part covers what to decide, why each decision matters, and how it becomes production code.

06 Foundations architecture

07 Colour

08 Typography

09 Spacing

10 Grid and layout

11 How foundations work together

CHAPTER 06

Understanding design foundations

Foundations are the small set of decisions that every pixel in your product inherits. They are the cheapest part of a design system to build and the most expensive part to get wrong.

READ THIS IF

You are about to define colour, type or spacing for a product, or you need to understand how a design decision becomes code without being re-typed by hand.

WHAT YOU'LL LEAVE WITH

01 A clear list of what counts as a foundation, and what doesn't.

02 An understanding of "blast radius": why foundation changes are high-impact.

03 The pipeline from a design decision to a token to production code, which the rest of Part 02 fills in.

06.1

What foundations are

A foundation is a **category of fundamental visual decision** that is not itself a component but is used by every component. Public systems differ in naming (Carbon calls them "elements", Material calls them "styles" and "foundations"), but the categories are remarkably consistent.[19][20]

CHAPTER 07
Colour
Palettes, ramps, semantic roles, themes, contrast.

CHAPTER 08
Typography
Typefaces, scale, line-height, roles, responsive type.

CHAPTER 09
Spacing
Base unit, scale, density, component and layout space.

CHAPTER 10
Grid and layout
Columns, gutters, breakpoints, containers, regions.

ALSO
Elevation
Shadows and layering (z-order) for surfaces.

ALSO
Shape
Corner radius and border widths.

ALSO
Motion
Durations, easing curves, reduced-motion behaviour.

ALSO
Iconography
Grid, stroke, sizes and optical alignment.

This edition covers the four foundations with the largest effect on product quality: colour, typography, spacing and layout. The others follow the same architecture and will be covered in a later edition.

06.2

Why foundations matter: blast radius

Every decision in a design system has a *blast radius*: the number of places it affects. A component property affects one component. A foundation affects all of them. That is why foundations are both the highest-leverage and the highest-risk part of the system.

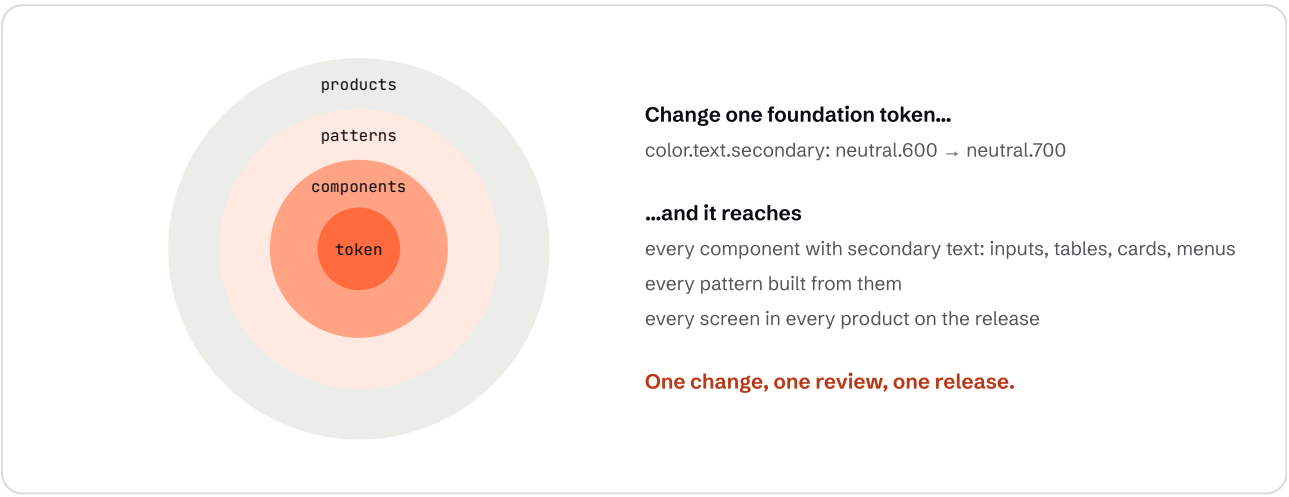


FIG 6.1 **Blast radius.** Here, a contrast fix applied once at the token layer reaches every product. The same mechanism spreads a bad decision just as efficiently, which is why foundations deserve the most careful review in the system.

Foundational decisions shape four outcomes:

Quality	Contrast, legibility and hierarchy are decided here. A type scale with too-small steps produces muddy hierarchy in every screen.
Consistency	A constrained set of values (say 12 spacing values instead of "whatever looks right") makes consistency the default rather than a review task.
Scalability	A semantic layer (07.4) makes themes, dark mode and new brands possible without touching components.
Delivery speed	Fewer choices mean faster decisions. "Which of these 12 spacing values?" is a quicker question than "how many pixels?"

06.3

From decision to token to code

The reason foundations can be changed safely is that they are never typed twice. A decision is captured once as a **design token**, a named value stored in a platform-agnostic format, and compiled into every platform's code by a build step.

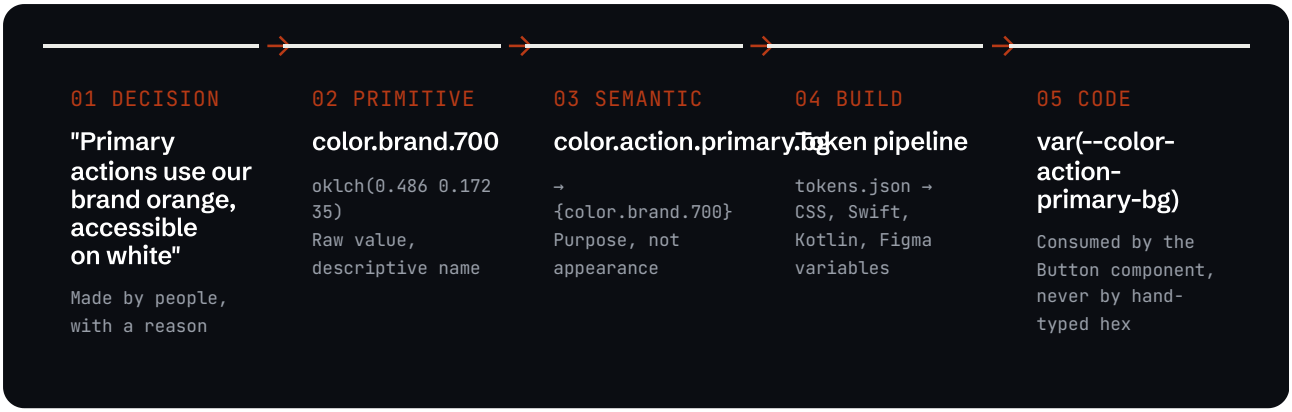


FIG 6.2 The foundation pipeline. A decision becomes a primitive token, then a semantic token, then platform code. Only step 01 involves judgement. Everything after it is automated, so a change at step 02 or 03 reaches every platform in one release.

A shared format

Until recently every tool used its own token format. In October 2025 the W3C Design Tokens Community Group published version 2025.10 of the Design Tokens Format Module, its first stable specification.[4][17] DOCUMENTED It defines a JSON format where each token has a `$value` and a `$type`, groups nest with dot-path names, and tokens reference each other with curly braces (`{color.brand.700}`). Note that it is a Community Group report, not a formal W3C Standard; it is nevertheless the format most token tools now converge on. This is what the Ledgerly tokens look like in it:

```
{
  "color": {
    "brand": {
      "500": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.645, 0.19, 37],
"hex": "#e95728"}},
      "600": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.565, 0.189, 37],
"hex": "#cc3c00"}},
      "700": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.486, 0.172, 35],
"hex": "#ab2800"}},
      "800": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.406, 0.146, 34],
"hex": "#861b00"}}
    },
    "neutral": {
      "50": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.975, 0.011, 260],
"hex": "#f3f7fe"}},
      "100": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.945, 0.018,
260], "hex": "#e6edf9"}},
      "700": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.485, 0.018,
260], "hex": "#595f69"}},
      "950": {"$type": "color", "$value": {"colorSpace": "oklch", "components": [0.255, 0.018,
260], "hex": "#1e232c"}}
    }
  },
  "semantic": {
    "action": {
      "primary": {
        "bg": {"$type": "color", "$value": "{color.brand.700}"},
        "text": {"$type": "color", "$value": "{color.neutral.50}"}
      }
    },
    "text": {
      "primary": {"$type": "color", "$value": "{color.neutral.950}"}
    }
  },
  "button": {
    "primary": {
      "background": {"$type": "color", "$value": "{semantic.action.primary.bg}"}
    }
  }
}
```

FIG 6.3 Design tokens in the DTCG 2025.10 format. Three tiers, generated from this playbook's data: primitives hold OKLCH values; semantic tokens reference primitives; the component token references a semantic token. Values were computed by the build script, not copied by hand.

PRACTICE **TOKENS FIRST, COMPONENTS SECOND**

Rule #4 of how we work, and the most important sequencing decision in a system. Components built before the token layer exists end up with hard-coded values that must be found and replaced later. Tokens built first make every component themeable from day one.

The next four chapters take each foundation in turn, from first principles to tokens. Chapter 11 then shows them working together in one component.

CHAPTER 07

Colour systems

Colour is the foundation teams get wrong most often, and the one where getting it right pays back fastest. This chapter goes from colour models to a production token architecture, and every value in it is computed, not guessed.

READ THIS IF

You are defining or repairing a palette, adding dark mode or a second brand, or you keep finding contrast failures in audits.

WHAT YOU'LL LEAVE WITH

01 Why OKLCH is the right model for building a system palette, and where it still needs checking.

02 A repeatable method for generating accessible colour ramps, with a complete worked palette.

03 A three-tier token architecture that makes themes, dark mode and multiple brands a configuration change.

07.1

Colour models: how colour is described

A colour model is a way of describing a colour with numbers. The model you build your palette in decides which questions are easy to answer. For a design system the most important question is *"is this lighter or darker than that, and by how much?"*, because that is what contrast, hierarchy and state changes depend on.

MODEL	DESCRIBES COLOUR AS	GOOD FOR	PROBLEM FOR DESIGN SYSTEMS
RGB	Amounts of red, green and blue light (0–255)	How screens work; final output	Not human-readable. Nobody can predict what `rgb(204 60 0)` looks like, or how to make it 10% lighter.
HEX	RGB written in base 16 (`#cc3c00`)	Compact; universal in tools and code	The same as RGB, and even less readable.
HSL	Hue (angle), saturation, lightness	Picking colours by intuition	"Lightness" is not perceived lightness: `hsl(60 100% 50%)` (yellow) and `hsl(240 100% 50%)` (blue) claim the same lightness but look nothing alike.
HSV / HSB	Hue, saturation, value (brightness)	Colour pickers in design tools	Same flaw as HSL; "value 100%" is a different perceived brightness for every hue.

MODEL	DESCRIBES COLOUR AS	GOOD FOR	PROBLEM FOR DESIGN SYSTEMS
OKLCH	Lightness (0–1), chroma (colourfulness), hue (angle)	Building palettes, ramps and themes	Some combinations fall outside what screens can show (07.2), and lightness is close to, but not identical with, WCAG contrast.

Perceptual versus non-perceptual

RGB, HSL and HSV are *non-perceptual*: equal numeric steps do not look like equal steps to the human eye. A **perceptual** colour space is designed so that numeric distance tracks perceived difference. Oklab, published by Björn Ottosson in 2020, is a perceptual space designed for exactly this.[21] OKLCH is the same space in polar form: lightness, chroma and hue. It is part of CSS Color Module Level 4 and supported natively in current browsers as `oklch()`.[22] STANDARD

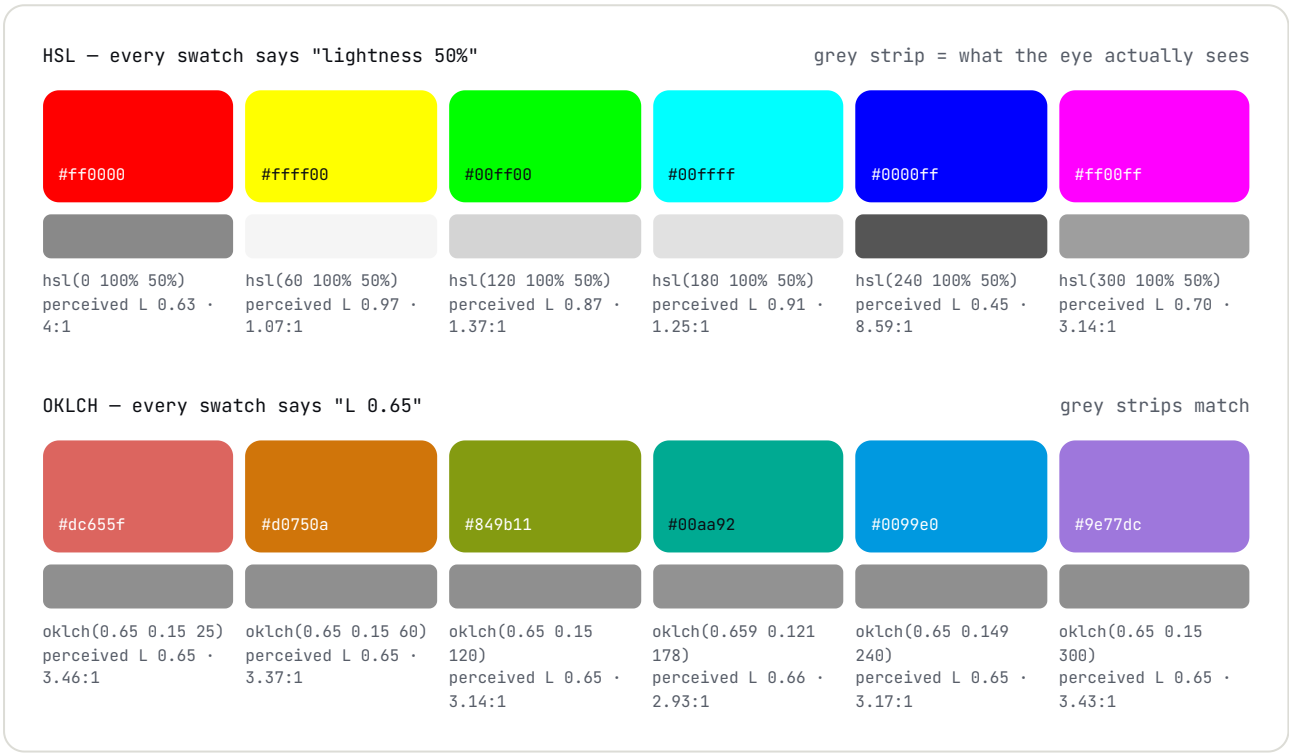


FIG 7.1 Same number, different reality. Top: six hues at HSL lightness 50%. Their perceived lightness (the grey strip, computed in Oklab) ranges from 0.45 to 0.97, and contrast on white from 1.07:1 to 8.59:1. Bottom: six hues at OKLCH lightness 0.65. The grey strips match. This is the whole argument for OKLCH in one picture.

Lightness, chroma and hue

Lightness L 0 → 1



How light the colour looks. The axis that drives contrast, hierarchy and states. Black is 0, white is 1.

Chroma C 0 → ~0.37



How colourful it is. 0 is grey. Unlike HSL saturation, the maximum depends on hue and lightness.

Hue H 0° → 360°



Which colour family. Every swatch in this row has L 0.70, so they all look equally light.

Why OKLCH is particularly useful for design systems

01 **Predictable lightness**

Step 700 can mean "this lightness" in every hue. So state changes, hierarchy and contrast behave the same across your whole palette.

02 **Independent axes**

Change hue (a new brand) without changing lightness, and so without breaking contrast. In HSL, rotating hue silently changes perceived lightness.

03 **Generated, not hand-picked, ramps**

Ramps become a formula (a lightness curve plus a chroma envelope), so new hues, brands and white-label themes come out consistent.

04 **Better gradients and mixing**

Interpolating in OKLCH avoids the grey, muddy midpoints of sRGB blends (07.7).

05 **Wide-gamut ready**

OKLCH can describe the more vivid colours of Display P3 screens. In sRGB values you can't even write them down.

06 **Native in CSS and the token spec**

`oklch()` is in CSS Color 4, and the DTCG 2025.10 format can store OKLCH components directly.[22][17] Tailwind CSS moved its default palette to OKLCH in v4.0.[23] DOCUMENTED

TWO CAVEATS THAT MATTER

1. Not every OKLCH colour can be displayed. Screens have a gamut, a range of colours they can physically produce. High chroma at very light or very dark lightness often falls outside sRGB. Tools must *gamut-map* (reduce chroma until the colour fits); our generator does this, and marks each chip where it happened.

2. OKLCH lightness is not WCAG contrast. WCAG 2 contrast is computed from relative luminance, a different formula.[24] Equal OKLCH lightness gives *similar* contrast across hues, not identical. You still validate every pairing. Figure 7.4 shows exactly how close it gets.

07.2

Building a palette

Step 1: Establish the primary colour, and separate brand from action

Most palettes start from a brand colour. The first discovery is usually that the brand colour, at its exact brand value, can't be used for text or for buttons with white labels. Layer One's own signal orange, `#ff6b3d`, is a good example: it has a contrast of **2.83:1 on white**, below the 4.5:1 WCAG 2.2 AA minimum for body text and below 3:1 for UI components. On the ink background it reaches 6.87:1. STANDARD

The answer is not to change the brand. It is to treat the brand colour as a *seed* for a ramp, and to choose ramp steps by role. The exact brand value is kept for large brand moments (illustration, the logo, dark-background accents), and a darker step from the same ramp is used for interactive elements on light backgrounds. In the palette below, the brand ramp is generated from `#ff6b3d` (`oklch(0.705 0.191 37)`), and `brand.700` becomes the primary action colour.

Step 2: Choose supporting hues by job, not by taste

A product palette needs far fewer hues than a brand palette. Most products need:

HUE	JOB	RULE
Brand / primary	Primary actions, selection, key brand moments	One hue; not reused for status
Neutral	Text, surfaces, borders, most of the interface	Low chroma, often tinted toward the brand or a cool hue
Danger (red)	Errors, destructive actions	Never the same hue as brand
Success (green)	Confirmation, positive status	Always paired with an icon or label
Warning (amber)	Caution, pending states	Dark text on amber, not white
Info / link (blue)	Links, informational messages, focus rings	May double as primary if brand is blue

Status colours must never carry meaning on their own. WCAG 2.2 success criterion 1.4.1 ("Use of Color") requires that colour is not the only visual means of conveying information,[12] so every status colour ships with an icon, a label or both. STANDARD

Watch for collisions between brand and status hues. Ledgerly's brand orange sits at hue 37° and its danger red at 25°, only 12° apart. That's a realistic situation for a warm brand, and it means colour alone could never tell "primary action" from "destructive action". The fix is structural, not chromatic: destructive buttons get an explicit label and confirmation, and errors always carry an icon.

Step 3: Build ramps on a fixed lightness curve

A **ramp** (also called a scale or tonal palette) is the set of lightness steps for one hue. The key move in OKLCH is to fix the lightness of each step *across all hues*: `blue.700`, `red.700` and `neutral.700` all have L ≈ 0.485. That makes the step number itself meaningful.

STEP	50	100	200	300	400	500	600	700	800	900	950
Lightness L	0.975	0.945	0.885	0.815	0.730	0.645	0.565	0.485	0.405	0.330	0.255
Chroma × peak	0.12	0.26	0.50	0.74	0.92	1.00	1.00	0.92	0.78	0.60	0.45
Typical role	Backgrounds, tints		Subtle fills, borders		Strong borders, icons, large accents		Text-safe accents, actions		Text, dark surfaces		

The curve is denser at both ends than in the middle. Interfaces need many very light tints (surfaces, hover backgrounds, selected rows) and several very dark values (text, dark-mode surfaces), but only a few mid-tones. Radix Colors takes a similar purpose-led approach with 12 steps, each assigned a job from app background to high-contrast text;[25] Material 3 generates tonal palettes from a key colour in its HCT colour space.[20] DOCUMENTED

Step 4: Shape chroma, then fit it to the gamut

Colourfulness should peak in the middle of the ramp and fall away toward both ends. Very light tints with high chroma look neon; very dark shades with high chroma look muddy. Our generator multiplies each hue's peak chroma by an envelope (the second row of the table above), then reduces chroma wherever the result falls outside sRGB. Lightness and hue are preserved, so contrast stays predictable.

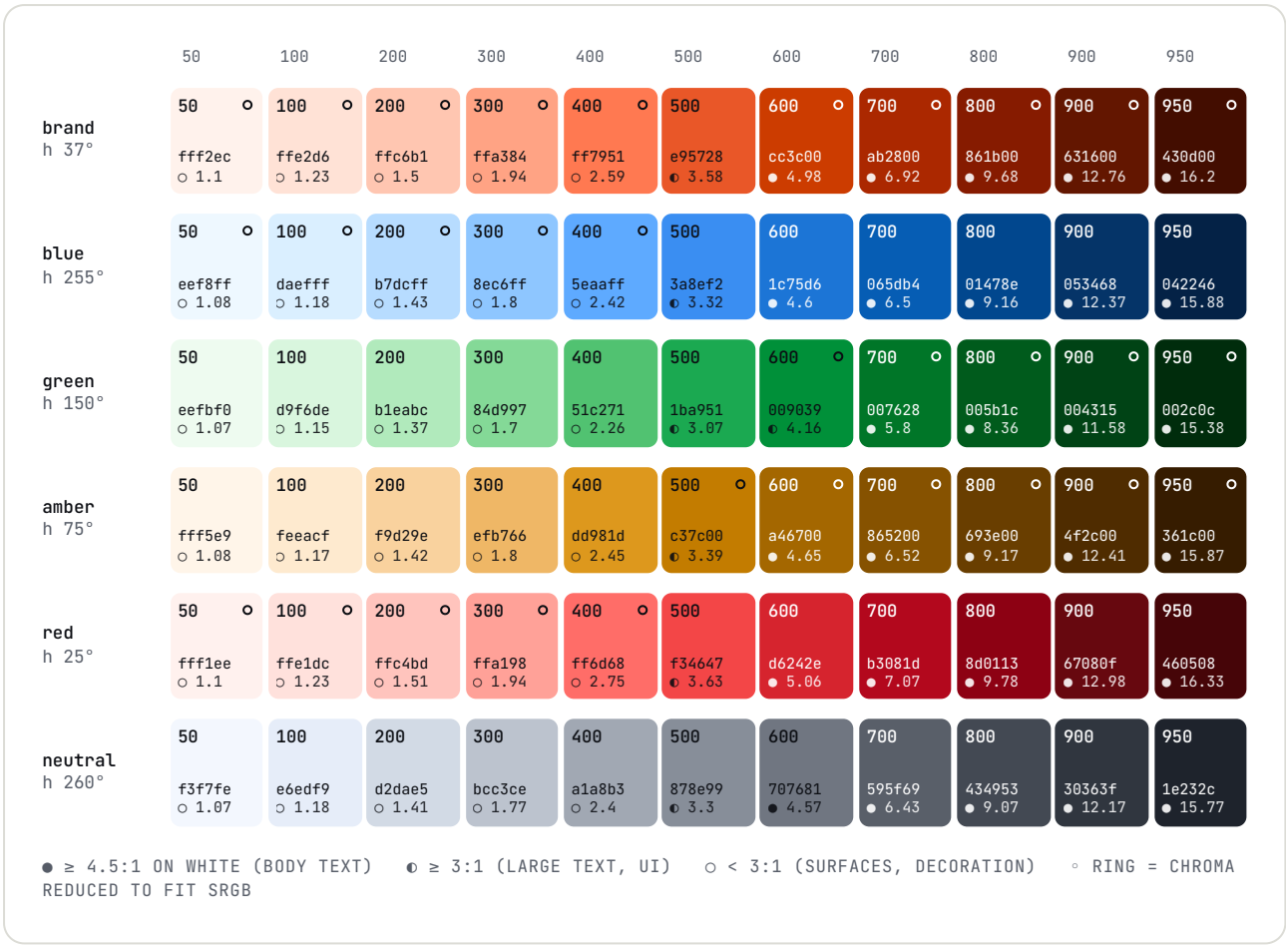


FIG 7.2 The Ledgerly palette: six ramps, 66 colours. Each chip shows its step, hex fallback and contrast against white. Every value is generated from three inputs: the lightness curve, the chroma envelope and one hue angle per ramp. A small ring marks chips where chroma was reduced to fit the sRGB gamut. Hover any chip in the HTML edition for its full OKLCH value.

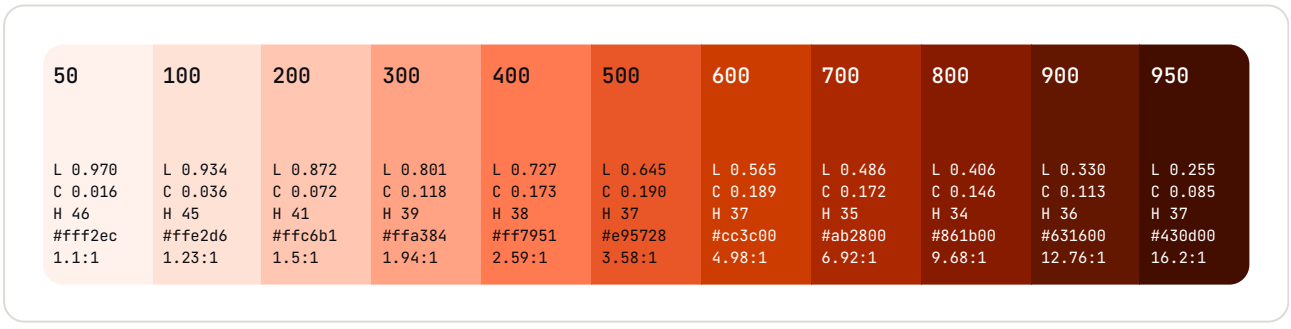


FIG 7.3 The brand ramp in detail. OKLCH lightness (L), chroma (C) and hue (H) for each step, with hex and contrast on white. Notice the lightness column climbing in even, predictable increments while chroma rises and falls.

Step 5: Validate contrast, and learn the pairing rules

WCAG 2.2 sets three thresholds you'll use constantly:[24][26] STANDARD

4.5:1

Minimum for body text (AA, SC 1.4.3). 7:1 for AAA (SC 1.4.6).

3:1

Large text: at least 18pt, or 14pt bold (about 24px / 18.66px).

3:1

Non-text: UI component boundaries, focus indicators, meaningful graphics (SC 1.4.11).



FIG 7.4 A fixed lightness curve produces near-identical contrast across hues. Six ramps, six lines that almost coincide. It isn't perfect: at step 600 green reaches 4.16:1 while red reaches 5.06:1, because WCAG luminance weights green more heavily than OKLCH lightness does. So the rule becomes "step 700 and darker pass 4.5:1 on white in every hue", which this palette guarantees, and anything else is checked individually.

Because lightness is fixed per step, contrast between two steps depends mostly on *how many steps apart* they are. That gives your team pairing rules they can apply without a contrast checker:

	50	100	200	300	400	500	600	700	800	900	950
50	—	1.1	1.3	1.6	2.2	3.1	4.3	6.0	8.4	11.3	14.7
100	1.1	—	1.2	1.5	2.0	2.8	3.9	5.5	7.7	10.3	13.4
200	1.3	1.2	—	1.3	1.7	2.3	3.2	4.6	6.4	8.6	11.2
300	1.6	1.5	1.3	—	1.4	1.9	2.6	3.6	5.1	6.9	8.9
400	2.2	2.0	1.7	1.4	—	1.4	1.9	2.7	3.8	5.1	6.6
500	3.1	2.8	2.3	1.9	1.4	—	1.4	1.9	2.8	3.7	4.8
600	4.3	3.9	3.2	2.6	1.9	1.4	—	1.4	2.0	2.7	3.5
700	6.0	5.5	4.6	3.6	2.7	1.9	1.4	—	1.4	1.9	2.5
800	8.4	7.7	6.4	5.1	3.8	2.8	2.0	1.4	—	1.3	1.7
900	11.3	10.3	8.6	6.9	5.1	3.7	2.7	1.9	1.3	—	1.3
950	14.7	13.4	11.2	8.9	6.6	4.8	3.5	2.5	1.7	1.3	—

● ≥ 7:1 (AAA TEXT) ● ≥ 4.5:1 (AA TEXT) ● ≥ 3:1 (LARGE TEXT, UI BOUNDARIES) BLANK < 3:1

FIG 7.5 Neutral ramp contrast matrix. Each cell is the contrast ratio between two neutral steps. In this palette, steps **five or more apart** always reach 3:1 (enough for borders and large text), and steps **seven or more apart** always reach 4.5:1 (body text). Rules like these are only possible on a perceptually even ramp.

WHAT ABOUT APCA?

APCA (the Accessible Perceptual Contrast Algorithm) is a newer contrast method that accounts for font size, weight and polarity, and it has been explored for future versions of WCAG.[27] Radix Colors, for example, tunes its text steps to APCA targets.[25] It is promising, but WCAG 2.x contrast remains the normative requirement for conformance today. Validate with WCAG 2 ratios, and use APCA as additional guidance for dark mode and thin type if you wish.

07.3

Neutrals: the colour you use most

In a typical product, neutrals cover far more of the screen than any other colour: text, backgrounds, surfaces, borders, dividers, disabled states. A carefully built neutral ramp does more for perceived quality than any brand colour.

Pure grey or tinted?

Pure greys (chroma 0) can feel cold and slightly "dead" next to a saturated brand colour. A very small amount of chroma (0.005–0.02 in OKLCH) tinted toward the brand hue (warm) or toward blue (cool)

How many neutral steps?

Use the same 11 steps as every other ramp so the pairing rules apply. In practice, light themes use about six neutral steps (50, 100, 200, 500, 700, 950) and dark themes about six more. Unused steps are cheap to keep and prevent ad-hoc greys later.

makes neutrals feel intentional. Ledgerly's neutral ramp uses chroma 0.018 at hue 260: a cool, barely visible tint. PRACTICE

COMMON MISTAKE

Light grey text

Low-contrast grey text for "secondary" information is the single most common accessibility failure on the web. WebAIM's 2025 scan found low-contrast text on 79.1% of the top million home pages.[9] Fix it at the token: `color.text.secondary` in Ledgerly maps to `neutral.700` (5.99:1 on the canvas), never to anything lighter.

07.4

Colour architecture: primitive, semantic and component tokens

A palette is a set of values. A colour *system* is the layer of meaning on top of it. Mature public systems (Atlassian, Carbon, Spectrum, Salesforce Lightning, Material) all separate raw values from usage, even though their names differ.[28][29][30][3] DOCUMENTED

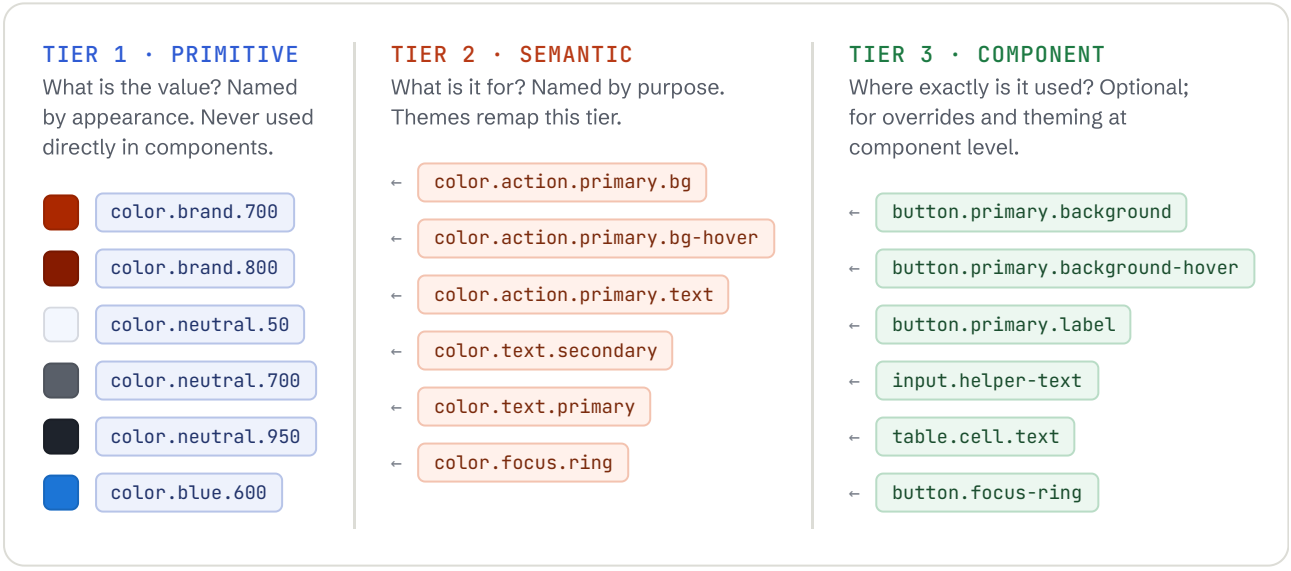


FIG 7.6 Three tiers of colour tokens. Read right to left: a component token references a semantic token, which references a primitive. Each row is one chain: `button.primary.background → color.action.primary.bg → color.brand.700`. Components only ever see tiers 2 or 3; the hex value lives in exactly one place.

Why separate raw values from usage?

Themes become a remap, not a rewrite

Dark mode changes which primitive each semantic token points to. No component changes.

Meaning survives refactoring

If danger moves from `red.700` to `red.800`, you change one reference, not every red in the product.

Decisions are reviewable

"Is `text.secondary` accessible?" is one check. "Are all the greys used for secondary text accessible?" is an audit.

The palette stays closed

Designers and engineers choose from roles, not from 66 swatches, so new ad-hoc colours stop appearing.

Do you need component tokens?

Component tokens add a level of indirection, and that has a cost: more tokens to name, document and maintain. Use them when **(a)** you support multiple brands or white-label theming that must restyle specific components, or **(b)** one component legitimately differs from the semantic default. Otherwise, components can consume semantic tokens directly. Many teams start with two tiers and add the third when theming demands it. PRACTICE

Brand tokens

In a multi-brand system, add a **brand tier** between primitives and semantics: each brand defines its own primitive ramps (or just its seed hues, from which ramps are generated), and the semantic tier references `brand.*` rather than a specific hue. Swapping the brand tier restyles every product. With OKLCH-generated ramps, a new brand needs only a seed colour; the lightness curve guarantees the same contrast behaviour.

Naming semantic colour tokens

Use a consistent grammar so people can guess names they haven't seen. Rule #3 of how we work is *"name things so people can guess them"*. A grammar that works across most products:

```
color.{category}.{role}.{variant?}.{state?}

color.bg.surface                    // backgrounds: canvas, surface, subtle, inverse
color.text.secondary                 // text: primary, secondary, disabled, inverse, link
color.border.strong                  // border: default, strong, focus
color.icon.subtle
color.action.primary.bg.hover       // interactive: role → part → state
color.feedback.danger.text          // status: danger, warning, success, info
```

SEMANTIC TOKEN	LIGHT THEME →	DARK THEME →
color.bg.canvas	neutral.50	neutral.950
color.bg.surface	neutral.50	neutral.900
color.bg.subtle	neutral.100	neutral.800
color.border.default	neutral.200	neutral.700
color.border.strong	neutral.500	neutral.500
color.text.primary	neutral.950	neutral.50
color.text.secondary	neutral.700	neutral.300
color.text.disabled	neutral.400	neutral.600
color.text.link	blue.700	blue.300
color.action.primary.bg	brand.700	brand.400
color.action.primary.bg-hover	brand.800	brand.300
color.action.primary.text	neutral.50	neutral.950
color.feedback.danger.text	red.700	red.300
color.feedback.danger.bg	red.100	red.900
color.feedback.success.text	green.700	green.300
color.feedback.success.bg	green.100	green.900
color.focus.ring	blue.600	blue.400

FIG 7.7 **Ledgerly's semantic colour tokens.** Each token points to a different primitive in each theme. This table *is* the theme: no component code is involved.

Background, surface, border, text and icon colours

CATEGORY	PURPOSE	RULE OF THUMB
Background	The canvas of the app and large regions.	Lightest steps in light mode; darkest in dark mode.
Surface	Cards, panels, menus, dialogs sitting on the background.	One step of separation from the background, plus a border or elevation.
Border	Dividers (decorative) and component boundaries (functional).	Decorative borders can be subtle; boundaries of inputs and controls need 3:1.
Text	Primary, secondary, disabled, inverse, link.	All except disabled must pass 4.5:1 on every surface they appear on.
Icon	Meaningful icons and decorative ones.	Meaningful icons need 3:1; often share the text tokens.

07.5

Interactive and state colours

Interactive elements need a colour for each state: default, hover, pressed (active), focus, selected and disabled. There are two common strategies:

STRATEGY A — STEP SHIFT

Move along the ramp

Hover moves one step darker (light mode) or lighter (dark mode): `brand.700 → brand.800`. Explicit, easy to check for contrast, and every state is a named token.

Default | Hover | Pressed | Focus

STRATEGY B — STATE LAYERS

Overlay a translucent layer

A semi-transparent layer of the content colour is placed over the component on hover or press. Material Design uses this approach. It works with any background, and fewer tokens are needed.

Default | + hover layer | + press layer

Focus deserves special care. A focus indicator is non-text content that must reach 3:1 against adjacent colours.[26] Use a dedicated `color.focus.ring` token (Ledgerly: `blue.600` in light mode, 4.28:1 on the canvas). Don't reuse the brand colour, which may not contrast with the component it surrounds. **Disabled** elements are exempt from contrast requirements under WCAG, but they should still be legible enough to be recognised. Never rely on disabled styling alone to explain why something can't be used.

07.6

Themes: light, dark, brand and white-label

With a semantic tier in place, a theme is simply a complete mapping from semantic tokens to primitives. The figure below shows the same mini-interface built from the same semantic tokens, rendered through two mappings.

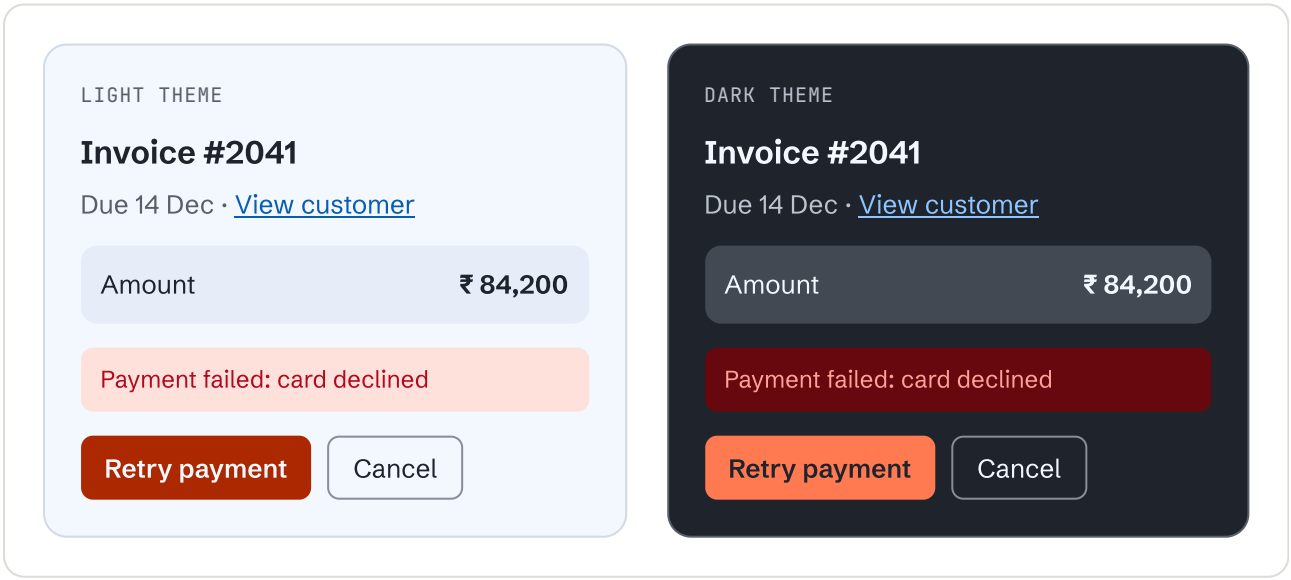


FIG 7.8 One component tree, two themes. Every colour on both cards comes from the semantic tokens in Figure 7.7. Nothing in the "component" changed between them; only the mapping did.

USE	PAIRING	WCAG MIN	LIGHT	DARK
Body text	`text.primary` on `bg.canvas`	4.5:1	14.67 pass	14.67 pass
Secondary text	`text.secondary` on `bg.canvas`	4.5:1	5.99 pass	8.89 pass
Links	`text.link` on `bg.canvas`	4.5:1	6.05 pass	8.76 pass
Primary button label	`action.primary.text` on `action.primary.bg`	4.5:1	6.44 pass	6.09 pass
Error message	`feedback.danger.text` on `feedback.danger.bg`	4.5:1	5.74 pass	6.68 pass
Success message	`feedback.success.text` on `feedback.success.bg`	4.5:1	5.02 pass	6.81 pass
Input border (non-text)	`border.strong` on `bg.canvas`	3:1	3.07 pass	4.77 pass
Focus ring (non-text)	`focus.ring` on `bg.canvas`	3:1	4.28 pass	6.51 pass
Disabled text (exempt)	`text.disabled` on `bg.canvas`	exempt	2.23	3.45

FIG 7.9 **Every semantic pairing, validated in both themes.** Computed by the build script from the actual token values. A theme isn't finished until this table passes, so make it a test that runs in CI.

Dark mode is not inversion

- **Don't use pure black.** Very dark neutral surfaces (Ledgerly: `neutral.950`, L 0.255) reduce glare and leave room for elevation.
- **Express elevation with lighter surfaces.** Shadows barely show on dark backgrounds. Raised surfaces step *lighter* (`neutral.900`, `neutral.800`).
- **Lighten, and often desaturate, accents.** Ledgerly's primary action moves from `brand.700` to `brand.400` in dark mode, with dark text on top (5.96:1).
- **Re-validate everything.** Pairings that pass in light mode can fail in dark mode, and vice versa.

Carbon, for example, ships four themes (two light, two dark) from a single set of semantic tokens. [29] DOCUMENTED

Multi-brand and white-label

SCENARIO	WHAT CHANGES	ARCHITECTURE
Light / dark	Semantic → primitive mapping	Two theme files over one palette
Multiple brands you own	Brand primitives; sometimes type and radius	A brand tier, plus light/dark per brand
White-label for customers	A small, controlled set of inputs (e.g. one seed colour, a logo)	Generate ramps from the seed in OKLCH; expose *only* those inputs, never raw tokens

PRACTICE **WHITE-LABEL SAFELY**

Let customers pick a seed colour, then generate their ramp with your lightness curve and map your semantic tokens to it. Because lightness is fixed per step, a customer's "brand.700" lands at the same lightness as yours, so contrast survives a choice you didn't make. Validate the generated pairings automatically and fall back to a darker step where a hue misses (as green does at 600).

07.7

Practical questions

Q

How many colours should a design system contain?

SHORT ANSWER — **Few hues, many steps, and fewer semantic tokens than you expect.**

A typical product needs one brand hue, one neutral, and three or four status hues: five to seven ramps. At 11 steps each, that's 55–77 primitives, but only 30–60 semantic tokens, and those are what people actually choose from. PRACTICE

Q

How many shades should a ramp have?

SHORT ANSWER — **Between 10 and 13 steps is the established range.**

Tailwind uses 11 (50–950), Radix 12, and Material's tonal palettes 13 tones.[23][25][20] DOCUMENTED Fewer than about 9 leaves gaps for states and dark mode; more than about 13 produces steps nobody can tell apart.

Q

Is there a maximum number of colours?

SHORT ANSWER — **There's no standard maximum, but there is a practical one: what your team can tell apart and remember.**

The symptom of too many colours isn't the palette size. It's semantic tokens with overlapping meaning (`text.muted`, `text.subtle`, `text.secondary`, `text.tertiary`). If two tokens can't be explained differently in one sentence, merge them.

Q

How do we stop colour proliferation?

- Components consume semantic tokens only; lint against raw colour values in product code.
- New semantic tokens need a reason ("which role doesn't exist yet?") and an owner's approval.
- Run the "unique colours in code" count from Chapter 03 every quarter. It should fall.

Q

How should we handle gradients?

SHORT ANSWER — **Interpolate in OKLCH and tokenise the stops.**

Blending in sRGB passes through a dull, greyish midpoint; OKLCH keeps the midpoint vivid and evenly bright. CSS lets you choose the interpolation space directly.



LINEAR-GRADIENT(IN SRGB, BLUE.600, AMBER.400)



LINEAR-GRADIENT(IN OKLCH, BLUE.600, AMBER.400)

Q

What about overlays and transparency?

SHORT ANSWER — **Use transparency for overlays and state layers, never for text.**

Scrims behind dialogs (`color.overlay.scrim → oklch(0.2 0.02 260 / 0.5)`) and state layers are good uses of alpha. Text with alpha is not: its contrast changes with whatever is behind it, so it can't be validated once. Give text solid tokens.

Colour system checklist

- [ ] Palette built in OKLCH from a fixed lightness curve
- [ ] Brand colour used as a seed; action colour is a text-safe step
- [ ] 5–7 ramps, 10–13 steps each, same step numbers across hues
- [ ] Gamut mapping applied; hex fallbacks generated, not hand-picked
- [ ] Neutral ramp with a deliberate tint (or a deliberate lack of one)
- [ ] Pairing rules published for designers
- [ ] Primitive and semantic tiers; components never reference primitives
- [ ] Token naming grammar documented
- [ ] State colours defined for every interactive role, including focus
- [ ] Light and dark themes as semantic mappings
- [ ] Every semantic pairing contrast-tested automatically in CI
- [ ] Status colours always paired with icon or text

CHAPTER 08

Typography systems

Most of any interface is text. A typography system isn't a font choice and a list of sizes. It is a set of roles, a scale, a rhythm and a set of rules that keep text readable on every screen and every platform.

READ THIS IF

You are choosing a typeface, defining a type scale, or your product's hierarchy feels muddy or inconsistent across screens.

WHAT YOU'LL LEAVE WITH

01 A method for building a type scale, from ratio to rounded, grid-aligned values.

02 A complete, tokenised product type scale you can adapt.

03 Guidance on how typography should differ across SaaS, enterprise, mobile, desktop and data-heavy products.

08.1

Choosing typefaces

For product interfaces, typeface selection is primarily a functional decision. Brand expression matters, but it is secondary to legibility at small sizes and coverage of everything the product needs to display.

CRITERION	WHAT TO CHECK	WHY IT MATTERS
Legibility at small sizes	Set 12–14px samples on real screens. Check x-height, open apertures, and distinguishable `Il1`, `O0`, `rn/m`.	Labels, tables and captions live at small sizes.
Weights and styles	At least regular, medium and semibold (or a variable weight axis); true italics if your content needs them.	Hierarchy depends on weight as much as size.
Numerals	Tabular (fixed-width) figures via `font-variant-numeric: tabular-nums`; clear distinction between 1 and 7.	Columns of numbers must align in tables and dashboards.
Language coverage	Every script you ship or plan to ship: Latin Extended, Cyrillic, Arabic, Devanagari…	A fallback font for one market breaks visual consistency.
Licensing	Web, app embedding, and number of products. Open-source (OFL) fonts remove most of the friction.	Licence terms can block a platform launch.
Performance	File size; variable font versus several static files; subsetting.	Fonts are often among the largest assets on first load.

SYSTEM FONTS

Platform UI fonts (San Francisco on Apple platforms, Roboto on Android, Segoe UI on Windows) are highly legible, free, cost nothing to load and feel native. They're the right default for many internal and enterprise tools. The trade-off is weaker brand distinctiveness.

HOW MANY FAMILIES?

One family for UI is usually enough. A second (a mono for code and data, or a display face for marketing) is common. A third is rarely justified in product UI. This playbook uses Schibsted Grotesk for everything and JetBrains Mono for values a machine reads.

Variable fonts

A variable font stores a continuous range of a design axis, most commonly weight, in one file.[31] Instead of choosing between four static weights, the system can use any weight between, say, 400 and 900. For a design system this means one file to load, exact weight tokens (`font.weight.semibold = 600`), and optical adjustments such as slightly heavier weights in dark mode.

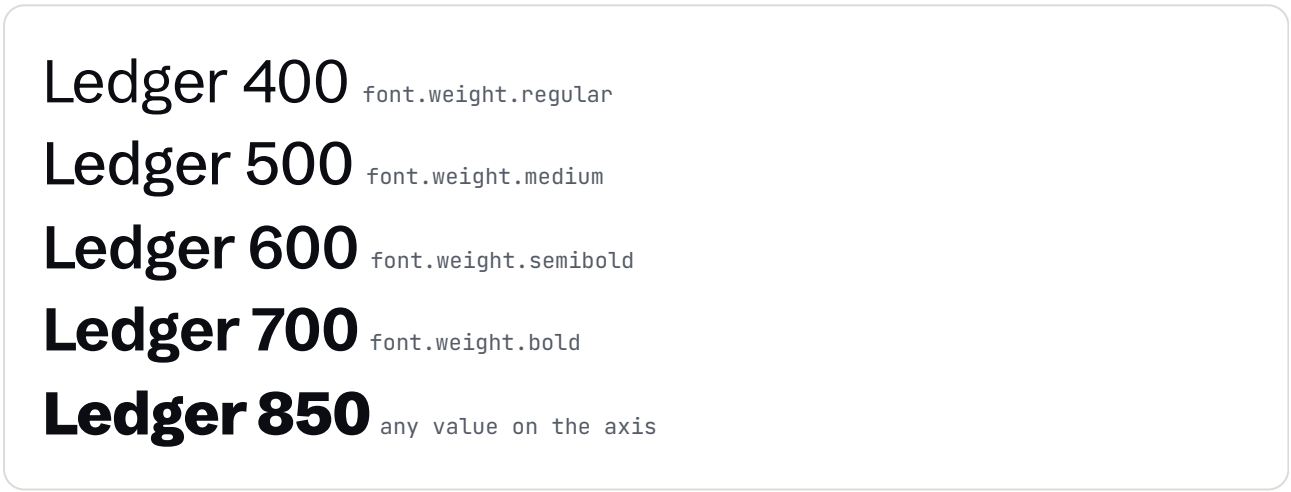


FIG 8.1 One variable font file, any weight. Schibsted Grotesk's weight axis runs from 400 to 900. Systems still tokenise a small set of named weights so that "any value" doesn't become "every value".

08.2

Type scale architecture

A type scale is the limited set of font sizes your product uses. The classical method is a **modular scale**: start from a base size (usually 16px, the browser default) and multiply by a constant ratio for each step up, divide for each step down.



FIG 8.2 Four modular scales from a 16px base. Computed values, steps –2 to +4. Small ratios (1.125) give many close sizes, suited to dense UI. Large ratios (1.618) jump quickly to display sizes, suited to editorial and marketing pages, and leave few usable sizes in between.

A pure modular scale is a starting point, not the answer. Product interfaces need several sizes clustered between 12 and 20px (labels, body, small headings), and only a few large sizes. A single ratio rarely provides both. Most mature systems therefore **start from a ratio and then adjust by hand**, rounding to whole pixels and to the line-height grid. Material 3's type scale, for example, is a curated set of 15 named styles rather than a strict geometric progression.[32] DOCUMENTED

Line-height: tighter as text grows

Line-height (leading) should *decrease* as a ratio as size increases. Body text needs about 1.4–1.6× for comfortable reading; large headings look loose above about 1.2×. Snap every line-height to your spacing grid (4px in this playbook) so text and components align vertically.

FONT SIZE	TARGET RATIO	RULE USED TO GENERATE THE SCALE
≤ 14px	≈ 1.45	line-height = round(size × ratio ÷ 4) × 4, the nearest multiple of 4px to the target. Small sizes end up slightly looser and large sizes slightly tighter, which is the direction you want. Headings use the tighter ratios at every size, so an 18px heading gets 24px, not the 28px of 18px body text.
15–20px	≈ 1.5	
21–32px	≈ 1.3	
33–48px	≈ 1.2	
> 48px	≈ 1.1	

Letter-spacing (tracking)

Typefaces are spaced for text sizes. At large sizes, the default spacing looks loose, so display and heading styles usually take slightly *negative* tracking (around −0.01 to −0.02em). At very small sizes, and in all-caps labels, slightly *positive* tracking improves legibility. Store tracking in `em` so it scales with size.

08.3

Roles and hierarchy: a complete product scale

Name type styles by **role**, not by size. "Heading large" survives a redesign; "28px" doesn't. Five role families cover almost every product interface:

DISPLAY	HEADING	BODY	LABEL	CAPTION
Hero numbers, marketing moments, empty-state headlines.	Page and section titles; the structure of a screen.	Paragraphs, descriptions, table cells, form values.	Buttons, tabs, form labels, navigation. Medium weight.	Metadata, timestamps, helper text, footnotes.

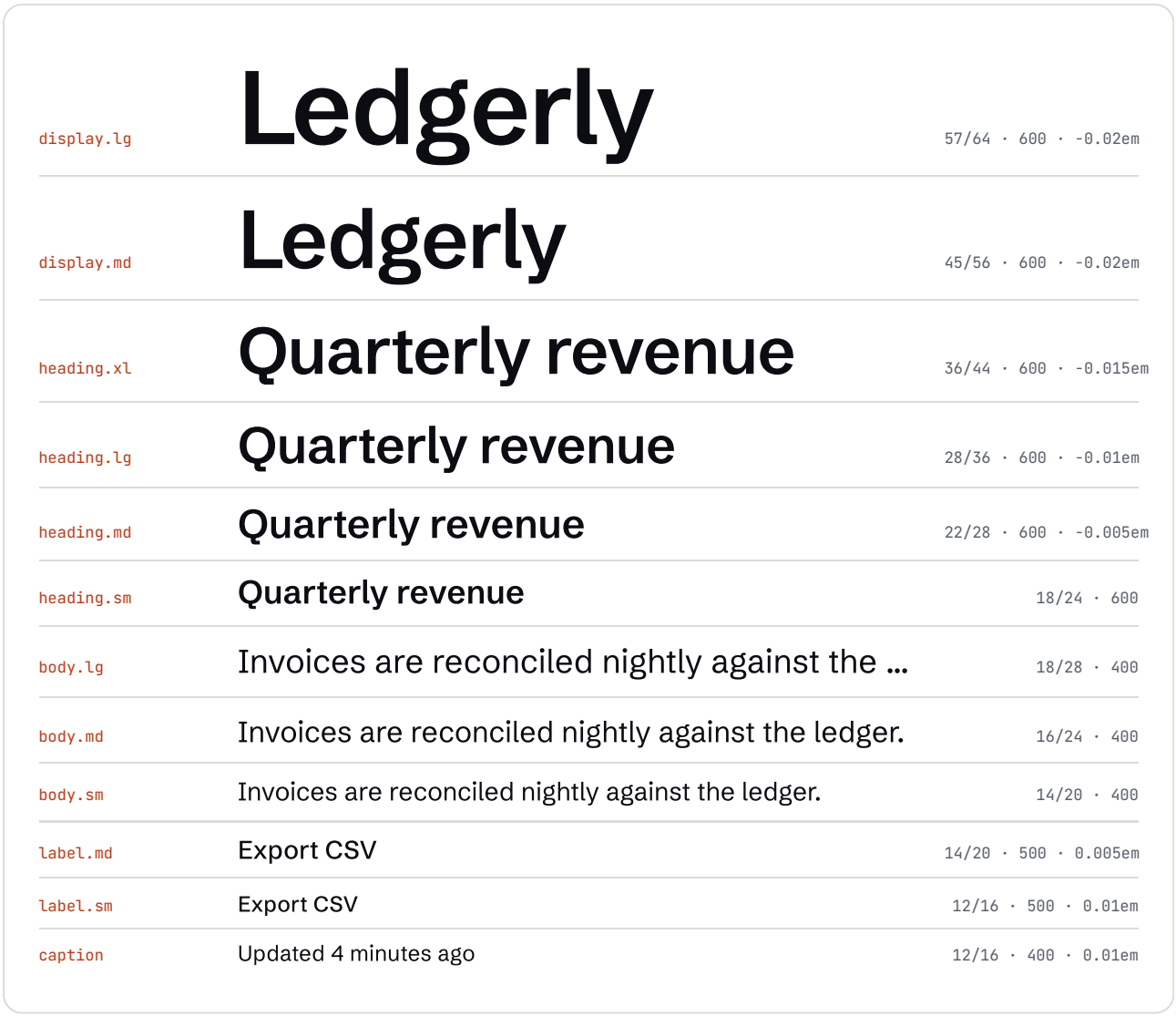


FIG 8.3 The Ledgerly product type scale. Twelve styles in five roles, set in Schibsted Grotesk at their actual sizes. Format: size/line-height in px · weight · tracking. Every line-height is a multiple of 4.

TOKEN	SIZE PX	LINE PX	RATIO	WEIGHT	TRACKING EM
`display.lg`	57	64	1.12	600	-0.02
`display.md`	45	56	1.24	600	-0.02
`heading.xl`	36	44	1.22	600	-0.015
`heading.lg`	28	36	1.29	600	-0.01
`heading.md`	22	28	1.27	600	-0.005
`heading.sm`	18	24	1.33	600	0
`body.lg`	18	28	1.56	400	0
`body.md`	16	24	1.5	400	0
`body.sm`	14	20	1.43	400	0
`label.md`	14	20	1.43	500	0.005
`label.sm`	12	16	1.33	500	0.01
`caption`	12	16	1.33	400	0.01

FIG 8.4 The scale as tokens. Each style becomes a composite typography token (family, size, line-height, weight, tracking). The ratio column shows line-height falling from about 1.5 to about 1.1 as size grows.

Hierarchy uses more than size

A common mistake is to create hierarchy with size alone, which inflates the scale. Weight, colour (`text.primary` vs `text.secondary`), case and spacing are equally strong signals and cost no new sizes.

× HIERARCHY BY SIZE ONLY

Invoices

Overdue

3 invoices totalling ₹2,40,000

Last reminder sent 2 days ago

✓ SIZE, WEIGHT, COLOUR AND SPACE

Invoices

OVERDUE

3 invoices totalling ₹2,40,000

Last reminder sent 2 days ago

The right-hand version uses fewer sizes but has a clearer hierarchy, because each level differs on two or more dimensions.

08.4

Responsive typography

Large display and heading sizes that work on a desktop overwhelm a phone. There are two approaches:

STEPPED

Different token values per breakpoint

`heading.xl` is 28px below 600px wide and 36px above. Simple, predictable, easy to spec in design tools.

FLUID

Interpolate between a minimum and maximum

Size scales smoothly with viewport width using CSS `clamp()`. Fewer jumps; popularised by tools such as Utopia.[33]

```
/* heading.xl: 28px at a 360px viewport → 36px at 1280px */
--type-heading-xl-size: clamp(1.75rem, 1.554rem + 0.87vw, 2.25rem);

/* slope     = (36 - 28) ÷ (1280 - 360) = 0.0087  → 0.87vw
   intercept = 28 - 0.0087 × 360  = 24.87px     → 1.554rem */
```

COMMON MISTAKE

Fluid type that ignores zoom

Font sizes set purely in viewport units (`font-size: 3vw`) don't grow when users zoom, which can fail WCAG 2.2 SC 1.4.4 (Resize Text, up to 200%).[12] STANDARD Always include a `rem` component in the fluid formula, as above, and only make display and heading sizes fluid. Body text should stay fixed and readable.

Native platforms add their own layer. iOS Dynamic Type and Android's font scaling let users choose larger text system-wide. Apple's guidelines encourage supporting Dynamic Type so text scales with the user's preference.[34] A design system for native apps should map each type token to the platform's scalable text style rather than to a fixed size.

08.5

Accessibility and readability

REQUIREMENT	WHAT IT MEANS FOR THE TYPE SYSTEM	BASIS
Contrast	4.5:1 for body text, 3:1 for large text (≥ 24px, or ≥ 18.66px bold). Defined in the colour tokens used by each text style.	STANDARD SC 1.4.3[24]
Resize	Text can scale to 200% without loss of content: use relative units and let containers grow.	STANDARD SC 1.4.4
Text spacing	Layouts must not break when users override line-height to 1.5×, paragraph spacing to 2×, letter spacing to 0.12em and word spacing to 0.16em. Avoid fixed-height text containers.	STANDARD SC 1.4.12[35]
Reflow	Content reflows to a 320 CSS px width without horizontal scrolling (except tables, maps and similar).	STANDARD SC 1.4.10[36]
Line length	Aim for about 45–75 characters per line for long-form text; cap reading widths with a `max-width` in `ch`.	DOCUMENTED Bringhurst[37]
Minimum sizes	Avoid body text below 14px in dense web UI and 16px on mobile web; reserve 11–12px for captions and labels.	PRACTICE

08.6

Typography across product categories

The same principles produce different scales for different products. The table summarises how we would tune a type system for each category. Platform defaults are documented; the rest is practice.

CATEGORY	BODY SIZE	SCALE RATIO	PRIORITIES	WATCH OUT FOR
SaaS web app	14–16px	~1.2	Clear headings for navigation; comfortable forms; 10–14 styles.	Marketing-site scales leaking into the app.
Enterprise software	14px (13px dense)	~1.125–1.2	Density and scannability; strong label/value contrast; long sessions.	Text so small it fails users with low vision; offer a comfortable density mode.
Mobile apps	iOS: 17pt body; Android: 16sp body large	Platform scale	Use platform text styles; support Dynamic Type and font scaling.[34][32] DOCUMENTED	Fixed-height rows that truncate when users enlarge text.
Desktop apps	13–15px	~1.125–1.2	Follow OS conventions; dense menus and panels; keyboard-heavy use.	Mixing OS and brand fonts inconsistently.
Responsive web	16px	1.2 mobile → 1.25–1.33 desktop	Fluid headings; fixed body; readable line length at every width.	Headings that wrap to five lines on phones.
Data-heavy dashboards	13–14px	~1.125	Tabular numerals; a "metric" display style; compact labels; mono for IDs.	Hierarchy lost in a sea of same-size numbers; use weight and colour.

IBM Carbon formalises a similar split with two type sets: a *productive* set for dense task-focused product UI and an *expressive* set for editorial and marketing moments.[38] DOCUMENTED If your product has both a dense app and a marketing surface, consider the same separation rather than one compromise scale.

08.7

Typography tokens

Typography uses two token levels. **Primitive** tokens hold individual values (families, sizes, weights, line-heights, tracking). **Composite** tokens bundle them into styles by role. The DTCG format defines a composite `typography` type for exactly this.[17]

```
// primitives
"font": {
  "family": { "sans": { "$type": "fontFamily", "$value": ["Schibsted Grotesk", "system-ui", "sans-serif"] } },
  "weight": { "semibold": { "$type": "fontWeight", "$value": 600 } }
},
// composite, by role
"type": { "heading": { "lg": {
  "$type": "typography",
  "$value": {
    "fontFamily": "{font.family.sans}", "fontWeight": "{font.weight.semibold}",
    "fontSize": { "value": 28, "unit": "px" }, "lineHeight": 1.29,
    "letterSpacing": { "value": -0.01, "unit": "em" }
  } } } }
```

Typography checklist

- [ ] One UI family (plus mono if needed), tested at 12–14px
- [ ] Tabular numerals available and used in tables
- [ ] 10–15 styles named by role, not size
- [ ] Line-heights snapped to the 4px grid
- [ ] Tracking defined in em for display and labels
- [ ] Responsive strategy chosen: stepped or fluid (with rem)
- [ ] Layouts survive 200% text resize and text-spacing overrides
- [ ] Reading widths capped at roughly 45–75 characters
- [ ] Native styles mapped to Dynamic Type / font scaling
- [ ] Composite typography tokens in design tool and code

CHAPTER 09

Spacing systems

Spacing is the foundation people notice least and feel most. It creates grouping, hierarchy and rhythm. Left to individual judgement, it is also the first thing to drift.

READ THIS IF

Your screens feel "slightly off" without anyone being able to say why, reviews keep debating padding, or you need to support compact and comfortable densities.

WHAT YOU'LL LEAVE WITH

01 A base unit and a spacing scale, with the reasoning behind both.

02 A clear model for component spacing versus layout spacing, and for density modes.

03 Spacing tokens, and the mistakes to avoid when introducing them.

09.1

Why spacing systems matter

Space does three jobs in an interface. It **groups** related things (the Gestalt principle of proximity: elements close together are read as belonging together). It creates **hierarchy**, because more space around something gives it more importance. And it sets **rhythm**, a repeated cadence that makes a screen feel calm and intentional.

When spacing is chosen per screen, each value is defensible on its own, but together they are noise: 13px here, 14px there, 22px somewhere else. Users don't consciously notice the numbers. They notice that the interface feels slightly unresolved.

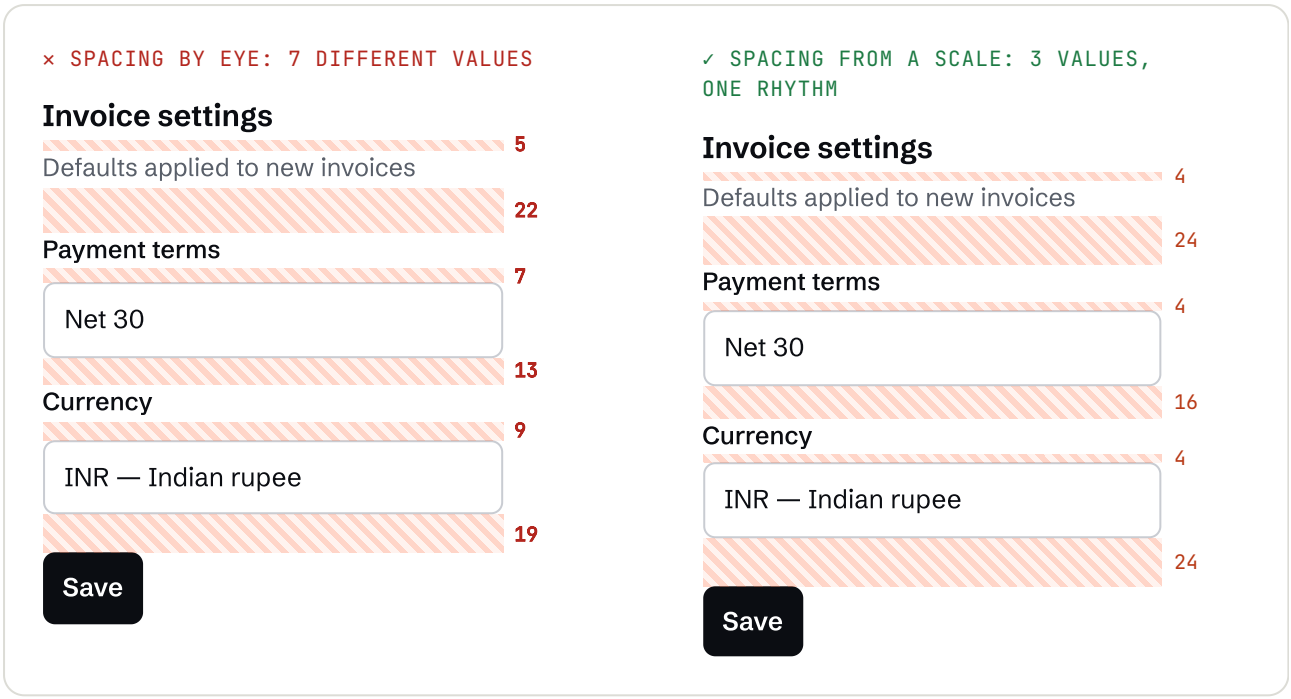


FIG 9.1 The same form, spaced by eye and by system. On the left, the gap between "Payment terms" and its field (7px) is almost the same as the gap between fields (13px), so labels float between inputs. On the right, three values make three clear relationships: *belongs to* (4), *next item* (16), *new group* (24).

09.2

Choosing a base unit

A spacing system starts with a base unit, and every spacing value is a multiple of it. The two common choices are **8px** and **4px**.

8PX BASE

Fewer, more distinct values

Easy to reason about; most screen sizes divide by 8. Used by many systems as the primary unit: Atlassian's spacing scale is built on an 8px base, with smaller half and quarter steps for tight UI.[39] DOCUMENTED

4PX BASE

Finer control for dense UI

Allows 12px, 20px and 28px, which matter for small components, icon alignment and line-heights. Material uses a 4dp baseline grid for type and icons alongside its 8dp layout grid.[40] DOCUMENTED

PRACTICE OUR RECOMMENDATION

Use a **4px base with an 8px rhythm**: every value is a multiple of 4, and most of the values people reach for are multiples of 8. You get the precision dense components need without the indecision of a scale where every value is available. Align line-heights (Chapter 08) to the same 4px grid so text and boxes share one vertical rhythm.

09.3

The spacing scale

A scale is the closed list of spacing values the system offers. It should be *dense at the small end*, where 4px is a visible difference, and *sparse at the large end*, where 64px versus 68px is not. A roughly geometric progression does this naturally.

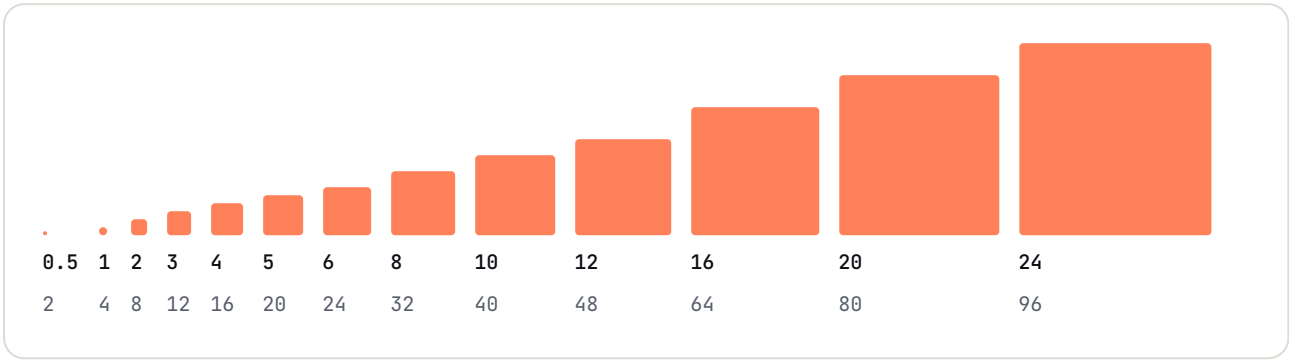


FIG 9.2 The Ledgerly spacing scale. Thirteen values from 2px to 96px, each drawn at actual size. Token names are multiples of the 4px base (`space.6` = 6 × 4 = 24px), so anyone can guess a token's value from its name and the other way round.

Naming spacing tokens

CONVENTION	EXAMPLE	PROS	CONS
Multiple of base	`space.6` = 24px	Guessable; easy maths; open-ended	Changing the base unit renames everything
Multiplier ×100	`space.300` = 3 × 8px (Atlassian)[39]	Room for fractional steps (`space.050`)	Values less obvious at a glance
Ordinal	`spacing-06` = 24px (Carbon)[19]	Compact; stable order	Must look up every value
T-shirt sizes	`space.md`	Friendly	Runs out of names (`xxxl`); hard to insert steps

Any convention works if it is consistent. Choose the one your engineers can read without a lookup table.

09.4

Component spacing and layout spacing

It helps to separate spacing *inside* components from spacing *between* them. Nathan Curtis describes three kinds: **inset** (padding inside a container), **stack** (vertical space between stacked elements) and **inline** (horizontal space between elements in a row).[41]

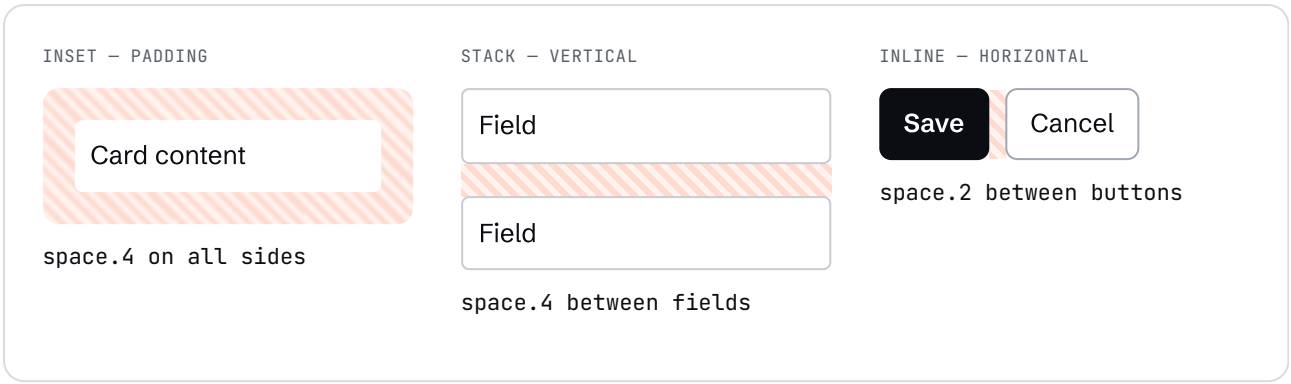


FIG 9.3 Inset, stack and inline. Naming the *kind* of space, not just the amount, makes intent explicit in specs and code reviews.

LEVEL	TYPICAL RANGE	EXAMPLES
Within a component	`space.1` – `space.4` (4–16px)	Icon-to-label gap, button padding, input padding, chip gap
Between related elements	`space.2` – `space.4` (8–16px)	Label to field, field to field, list items
Between groups	`space.6` – `space.8` (24–32px)	Form sections, card grid gaps, toolbar to content
Between sections	`space.10` – `space.16` (40–64px)	Page sections, settings groups
Page margins	`space.4` – `space.20` (16–80px)	Responsive; see Chapter 10

The proximity rule: space *within* a group must always be clearly smaller than space *between* groups. A useful heuristic is to step at least two scale steps between levels (8 → 16 → 24/32). That keeps relationships unambiguous even when designers make reasonable local choices. PRACTICE

09.5

Density: compact and comfortable

Density is how much information fits in a given area. Different users and tasks need different density: an analyst scanning 200 rows wants compact; a first-time user filling a form wants comfortable. Systems handle this with **density modes** that remap component sizes and spacing, not with separate components.

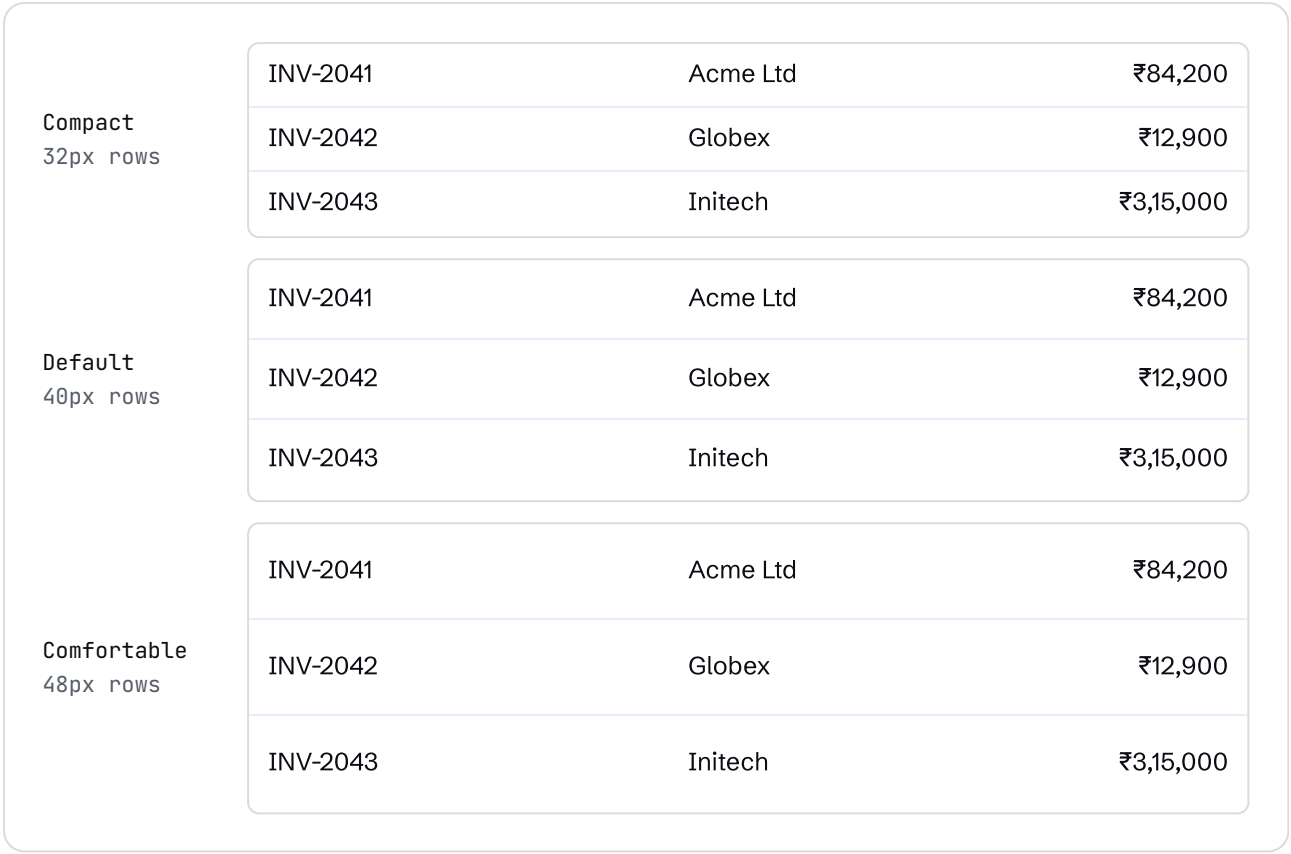


FIG 9.4 One table, three densities. Same type, same columns; only the row-height and padding tokens change. Compact fits 50% more rows than comfortable in the same height.

Density has a floor set by accessibility. WCAG 2.2 SC 2.5.8 (Target Size, Minimum, level AA) requires pointer targets of at least 24 × 24 CSS px, with some exceptions such as adequately spaced targets and inline links.[42] STANDARD Platform guidelines go further for touch: Apple recommends at least 44 × 44pt for controls,[43] and Material recommends 48 × 48dp touch targets.[40] DOCUMENTED Compact modes are for pointer-driven desktop use; never ship them as the default on touch devices.

TOKEN	COMPACT	DEFAULT	COMFORTABLE
`size.control.height`	32px	40px	48px
`space.control.inset-x`	8px	12px	16px
`space.table.row-y`	4px	8px	12px
`space.form.stack`	12px	16px	24px

09.6

Responsive spacing and spacing tokens

Component spacing should generally **not** change with screen size: a button's padding is the same on a phone and a desktop. **Layout** spacing should: page margins, section gaps and grid gutters grow with available space. Encode that split in tokens:

```
/* Tier 1: primitive scale (never changes with breakpoint) */
--space-2: 8px;  --space-4: 16px;  --space-6: 24px;  --space-10: 40px;  --space-16: 64px;

/* Tier 2: semantic layout spacing (responsive) */
--space-page-margin:  var(--space-4);     /* 16px on compact screens   */
--space-section-gap:  var(--space-10);
@media (min-width: 840px) {
  --space-page-margin: var(--space-6);   /* 24px on medium and up      */
  --space-section-gap: var(--space-16);
}

/* Tier 2: semantic component spacing (density-aware, not responsive) */
--space-control-inset-x: var(--space-3);
```

Common spacing mistakes

MISTAKE

Off-scale "just this once" values

Each one is harmless; fifty are a second, invisible spacing system. Lint for raw pixel values in spacing properties.

MISTAKE

Equal spacing everywhere

Using 16px for everything removes the proximity signal. Groups disappear. Vary space deliberately by level.

MISTAKE

Margins on components

Components that carry their own outer margin collide unpredictably. Let parents own the space between children (CSS `gap`).

MISTAKE

Ignoring line-height in vertical spacing

A 16px gap between two text blocks with loose line-heights looks larger than 16px. Align text to the same 4px grid so measured and perceived space match.

Spacing checklist

- [ ] Base unit chosen (4px with an 8px rhythm recommended)
- [ ] Closed scale of ~12–14 values, dense small, sparse large
- [ ] Token names guessable from values
- [ ] Inset / stack / inline vocabulary in specs
- [ ] Proximity rule: within-group < between-group, by 2+ steps
- [ ] Density modes remap tokens; touch targets ≥ 24px (AA), 44–48 on touch
- [ ] Component spacing fixed; layout spacing responsive
- [ ] Lint against raw spacing values

CHAPTER 10

Grid and layout systems

Grids are how a product decides where things go on every screen size before anyone designs a single screen. In modern products, layout is less about a column grid on a page and more about regions, containers and how they adapt.

READ THIS IF

You are defining breakpoints, designing an app shell or dashboard, or your screens break in different ways on every device.

WHAT YOU'LL LEAVE WITH

01 The anatomy of a grid (columns, gutters, margins, containers) and the numbers for each.

02 A breakpoint system grounded in published systems.

03 A layout model for SaaS dashboards and complex enterprise applications.

10.1

Grid fundamentals

A layout grid divides the available width into **columns** separated by **gutters**, inside **margins**, within a **container**. Content spans whole columns (and the gutters between them), so edges align across the screen without anyone measuring.

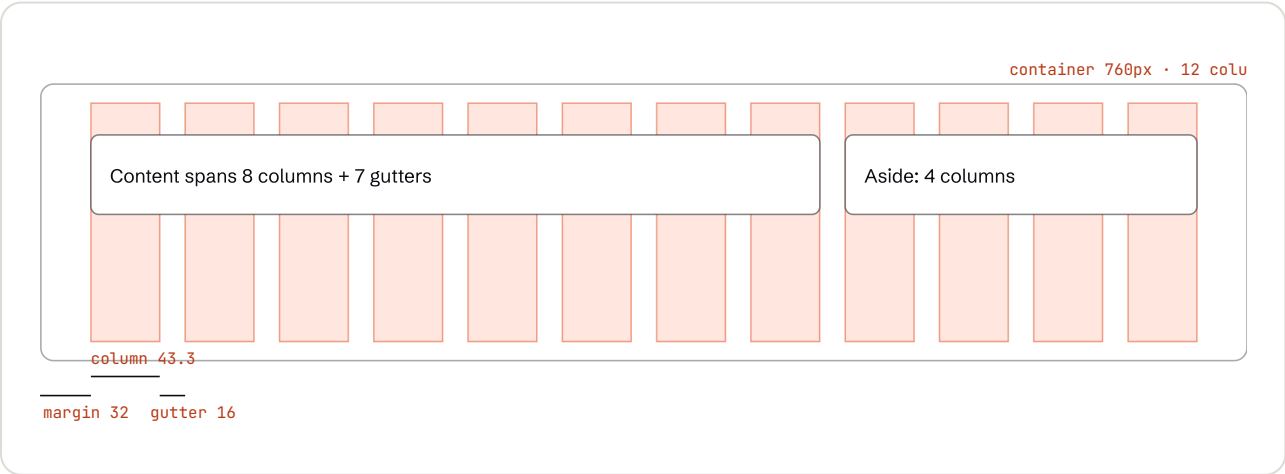


FIG 10.1 Anatomy of a 12-column grid. A 760px container with 32px margins and 16px gutters leaves 41.3px columns. Twelve divides evenly into 2, 3, 4 and 6, which is why it's the most common column count for desktop layouts.

PART	DEFINITION	GUIDANCE
Columns	Vertical divisions content snaps to.	4 on phones, 8 on tablets, 12 on desktop is the common pattern.
Gutters	Fixed space between columns.	Use spacing tokens (16-24px). Gutters stay fixed; columns flex.
Margins	Space between the grid and the screen edge.	Grow with screen size: 16px on phones up to 32–80px on large screens.
Container	The box the grid lives in; may have a maximum width.	Cap reading layouts (~1200–1440px); let data-dense apps go full width.

10.2

Breakpoints and responsive grids

Breakpoints are the widths at which the layout changes structure. Set them where *your layout* needs to change, informed by common device classes, rather than by specific devices, which change every year.

Material 3 window size classes ■ DOCUMENTED

CLASS	WIDTH (DP)
Compact	< 600
Medium	600–839
Expanded	840–1199
Large	1200–1599
Extra-large	≥ 1600

Source: Material 3 layout guidance.[44]

IBM Carbon 2x Grid breakpoints ■ DOCUMENTED

NAME	FROM (PX)	COLUMNS
sm	320	4
md	672	8
lg	1056	16
xlg	1312	16
max	1584	16

Source: Carbon 2x Grid.[45] Carbon's 16-column grid suits dense product UI.

Two mature systems, two different sets of numbers, and both work. The lesson is not to copy either set. Pick a small number of breakpoints (four or five), give them names rather than device labels, and tokenise them so design and code agree. Ledgerly uses Material's classes with 4, 8 and 12 columns.

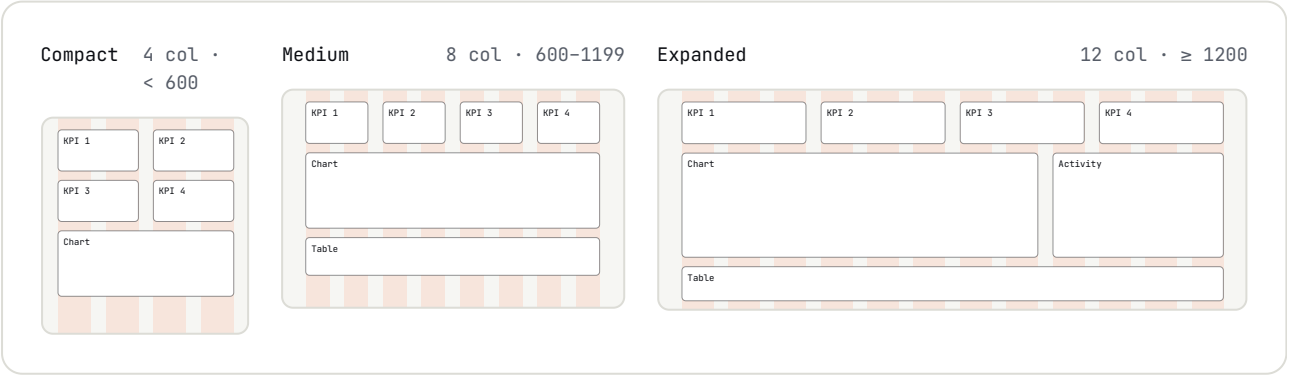


FIG 10.2 The same dashboard at three breakpoints. The content and its order don't change; the number of columns each block spans does. KPIs go from 2-up on a phone to 4-up on desktop; the activity panel only earns its own column when there is room for it.

10.3

Fluid, fixed and hybrid layouts

FLUID

Columns stretch with the viewport

Uses all available space. Right for data-dense apps, but long lines of text become unreadable on wide screens.

FIXED

Container has a set width

Predictable, easy to design, but wastes space on large screens and needs separate layouts for small ones.

HYBRID (RECOMMENDED)

Fluid up to a maximum

Columns flex between breakpoints; the container caps at a maximum width for reading layouts. Apps can opt out of the cap.

Content width

Grids decide where things *can* go; content width decides how wide reading content *should* be. Line length for prose is best kept around 45–75 characters,[37] which means body text should rarely span all 12 columns of a wide screen. Set content-width tokens independently of the grid:

TOKEN	VALUE	USED FOR
`layout.width.prose`	68ch	Articles, help content, long descriptions
`layout.width.form`	560px	Single-column forms and settings
`layout.width.dialog.md`	560px	Standard dialogs
`layout.width.page`	1440px	Max container for marketing and reading layouts
`layout.width.app`	none	Data-dense application views, full width

10.4

Composing a SaaS dashboard

Most SaaS products are not pages on a grid. They are an **app shell** (persistent navigation and chrome) around a **content region** that has its own grid. Defining the shell's regions as part of the system removes a whole category of per-team layout decisions.

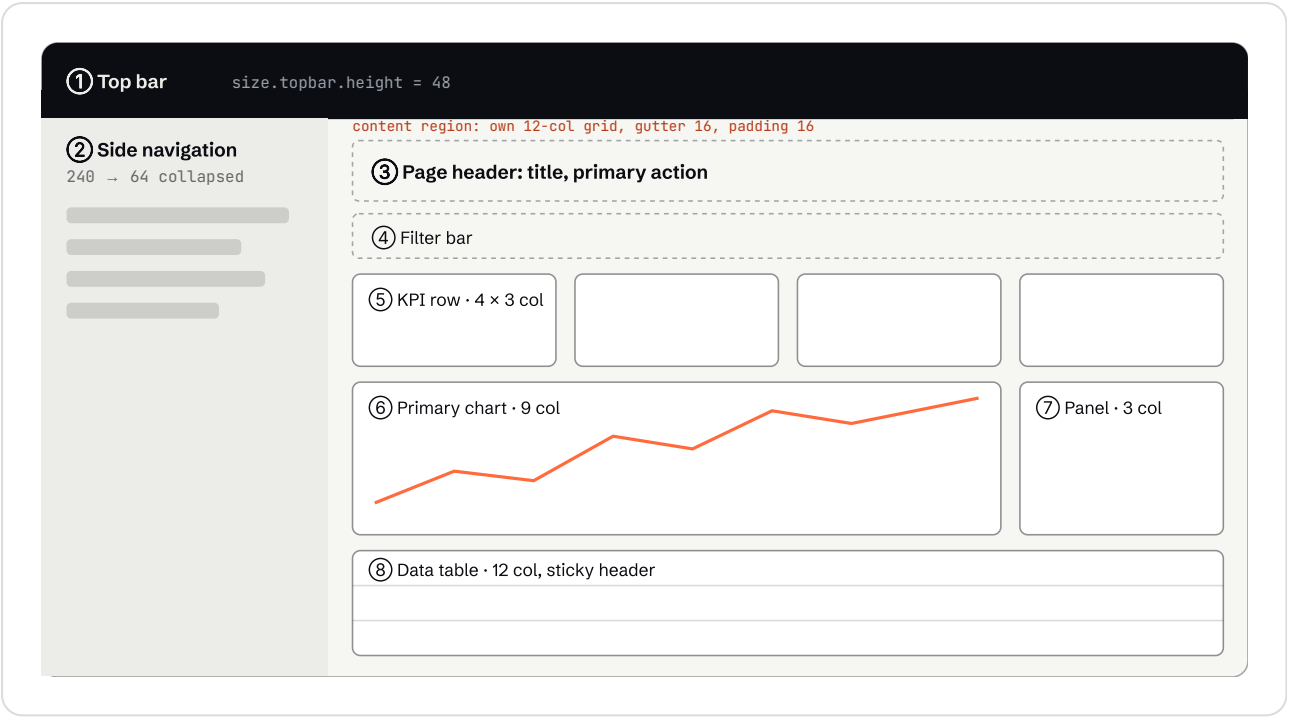


FIG 10.3 Regions of a SaaS dashboard. Regions ①–② belong to the shell and are identical on every page. Regions ③–④ follow a page-template pattern. Regions ⑤–⑧ are laid out on the content region's own 12-column grid. Each region's size is a token, not a per-page choice.

Dashboard composition rules

01 **Most important information top-left**

For left-to-right languages, readers scan from the top-left. Put the KPI or chart that answers "is everything OK?" first.

02 **Equal-height rows**

Cards in the same row share a height, so the grid stays visible and the eye can compare them.

03 **Spans from a short list**

Allow 3, 4, 6, 8, 9 and 12 columns on desktop. Avoid 5 and 7, which create misaligned edges.

04 **Mirror for right-to-left**

Use logical properties (`inline-start`, not `left`) so Arabic and Hebrew layouts mirror correctly from the same tokens.

10.5

Desktop, mobile and alignment

Desktop and mobile are different layouts, not one layout scaled

On mobile, side navigation becomes a bottom bar or a drawer; multi-pane views become single screens with back navigation; tables become lists or cards. Decide these transformations once, as patterns, and every product inherits them.

Align to columns at the page level, to spacing inside components

Page regions and cards align to grid columns. Inside a card, elements align to each other using spacing tokens; a button doesn't need to snap to a column. Trying to grid everything produces rigid, awkward components.

REFLOW, AND ITS EXCEPTIONS

WCAG 2.2 SC 1.4.10 requires content to reflow at a width of 320 CSS px without two-dimensional scrolling, except for content that requires a two-dimensional layout, such as data tables, maps and diagrams. [36] STANDARD Plan the 320px layout of every pattern; scroll horizontally only inside a table, never the whole page.

10.6

Layout for complex enterprise applications

Enterprise applications push past the single-grid model. The layout system needs a vocabulary for structures that page grids don't describe:

Multi-pane layouts

List-detail and supporting-pane layouts that show two or three panes side by side on large screens and collapse to one on small. Material documents these as canonical layouts. [44]

Resizable and collapsible panes

Users of complex tools expect to resize panels. Define minimum and maximum widths as tokens, and remember user preferences.

Nested grids and container queries

A component placed in a narrow pane should adapt to *its container*, not the viewport. CSS container queries make components responsive to the space they're given.

Sticky regions and dense tables

Sticky headers, frozen first columns and horizontal scrolling inside tables keep context while users work through large datasets.

Layout tokens

```
"breakpoint": { "medium": "600px", "expanded": "840px", "large": "1200px", "xlarge": "1600px" },
"grid":       { "columns": { "compact": 4, "medium": 8, "expanded": 12 },
               "gutter": "{space.4}", "margin": { "compact": "{space.4}", "expanded": "{space.6}"
} },
"layout":     { "sidebar": { "width": "240px", "collapsed": "64px" }, "topbar": { "height": "48px"
},
               "width": { "prose": "68ch", "form": "560px", "page": "1440px" } }
```

A note on breakpoints as tokens: CSS custom properties can't be used inside media query conditions, so breakpoint tokens are compiled to constants (Sass variables, JS constants, or `@custom-media` where supported) rather than to `var()`.

Grid and layout checklist

- [ ] 4–5 named breakpoints, tokenised for design and code
- [ ] Columns per breakpoint (e.g. 4 / 8 / 12)
- [ ] Gutters and margins from the spacing scale
- [ ] Hybrid container: fluid with a max width for reading
- [ ] Content-width tokens for prose, forms and dialogs
- [ ] App shell regions defined and sized by tokens
- [ ] Mobile transformations defined as patterns
- [ ] Every pattern works at 320px (tables excepted)
- [ ] Logical properties for right-to-left support
- [ ] Container queries for components in panes

CHAPTER 11

How foundations work together

Colour, type, spacing and layout are not four separate deliverables. They are one system, and the best way to see it is to follow a single component from the first decision to the line of code that ships.

READ THIS IF

You want to see the whole architecture in one place, or you need to explain to engineering how foundations become components.

WHAT YOU'LL LEAVE WITH

01 One component traced through five layers: decision, token, component, pattern and production.

02 A clear picture of what changes, and what doesn't, when a foundation changes.

03 A foundations readiness checklist to close out Part 02.

11.1

One button, five layers

The primary button is the most-used interactive component in most products. Here is every foundation it depends on, and how each one gets from a decision to the screen.

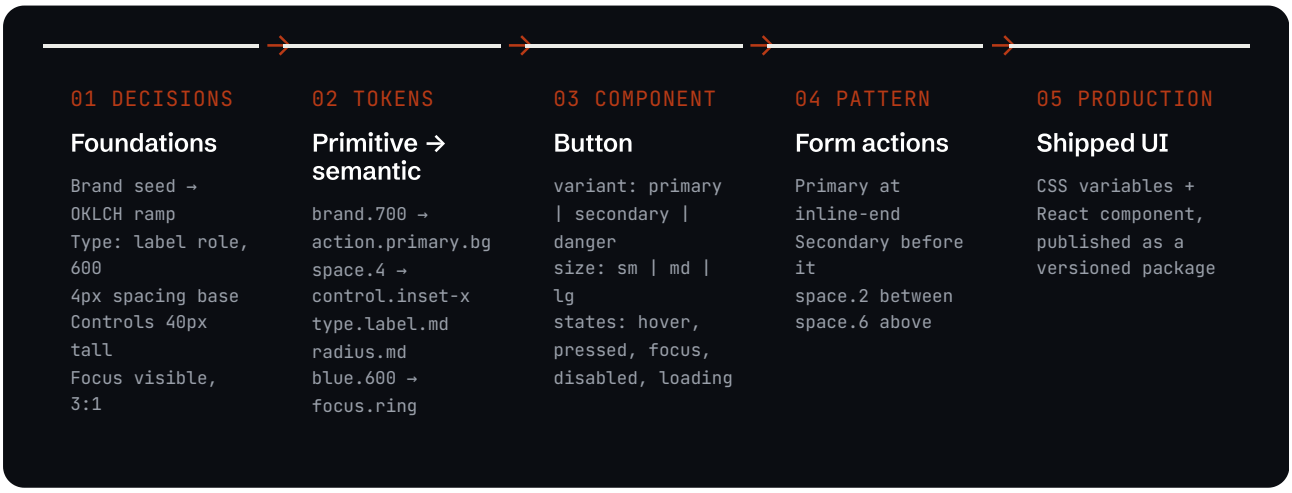
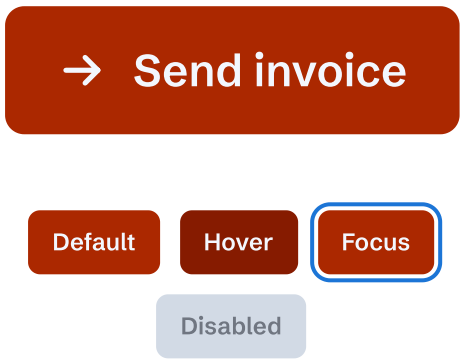


FIG 11.1 **The foundation-to-production chain.** Each layer only knows about the layer directly beneath it. The button doesn't know that `brand.700` exists, and the form doesn't know the button's padding. That ignorance is the point: it's what makes each layer changeable.

The component, annotated



PROPERTY	TOKEN	RESOLVES TO (LIGHT)
Background	color.action.primary.bg	brand.700
Background, hover	color.action.primary.bg-hover	brand.800
Label colour	color.action.primary.text	neutral.50 · 6.48:1
Label type	type.label.md	14/20 · 600
Height	size.control.height	40px (default density)
Padding x	space.control.inset-x	space.4 · 16px
Icon gap	space.2	8px
Radius	radius.control	6px
Focus ring	color.focus.ring	blue.600 · 2px, offset 2px

FIG 11.2 Every visual property of the button is a token. Shown at 1.6× scale. Nine properties draw on all four foundations: colour (four tokens), typography (one), spacing and sizing (three) and shape (one). The label contrast, 6.48:1, was computed from the actual token values in Chapter 07.

11.2

Into production

The token pipeline compiles the token source into each platform's format. For the web, that's CSS custom properties, which the component consumes. Notice that the component code contains **no raw values at all**.

```css
/* tokens.css: generated, never hand-edited
*/
:root {
  --color-brand-700: oklch(0.486 0.172 35);
  --color-brand-800: oklch(0.406 0.146 34);
  --color-action-primary-bg:
    var(--color-brand-700);
  --color-action-primary-bg-hover:
    var(--color-brand-800);
  --space-control-inset-x: var(--space-4);
  --size-control-height: 40px;
}
[data-theme="dark"] {
  --color-action-primary-bg:
    var(--color-brand-400);
}
[data-density="compact"] {
  --size-control-height: 32px;
  --space-control-inset-x: var(--space-2);
}
```

```css
/* button.css: consumes semantic tokens only
*/
.button {
  height: var(--size-control-height);
  padding-inline: var(--space-control-inset-
x);
  gap: var(--space-2);
  border-radius: var(--radius-control);
  font: var(--type-label-md);
}
.button[data-variant="primary"] {
  background: var(--color-action-primary-bg);
  color: var(--color-action-primary-text);
}
.button[data-variant="primary"]:hover {
  background:
    var(--color-action-primary-bg-hover);
}
.button:focus-visible {
  outline: 2px solid var(--color-focus-ring);
  outline-offset: 2px;
}
```

Native platforms receive the same tokens as Swift and Kotlin constants from the same source file. The token source is the contract; each platform's components are implementations of it.

11.3

When a foundation changes

The real test of the architecture is change. For each kind of change, here is which layers are touched, and how many are not.

CHANGE	PRIMITIVES	SEMANTIC	COMPONENTS	PRODUCTS	EFFORT
Rebrand: new brand hue	●	○	○	○	Change one seed; ramps regenerate; validate contrast; release.
Add dark theme	○	●	○	○	One new semantic mapping; validate pairings; release.
Fix secondary-text contrast	○	●	○	○	One reference change; every product fixed on upgrade.
Add compact density	○	●	◖	○	Density mapping; check components with fixed internal sizes.
New type scale	●	◖	○	◖	Values change; products check layouts that depended on old sizes.

CHANGE	PRIMITIVES	SEMANTIC	COMPONENTS	PRODUCTS	EFFORT
White-label for a customer	●	○	○	○	Generate a brand tier from the customer's seed at runtime or build time.

● changed · ◖ may need review · ○ untouched. These outcomes hold *only* if components consume semantic tokens exclusively. A single hard-coded hex in a component breaks the chain for that component.

THE WHOLE OF PART 02 IN ONE SENTENCE

Decide once in OKLCH, on a 4px grid, in a named type scale; encode the decisions as primitive and semantic tokens; let components consume only semantic tokens. Then every future change is a configuration change.

11.4

Foundations readiness checklist

Before building components at scale, confirm the foundations are ready. Each item links back to the chapter that explains it.

Colour ch. 07

- [ ] OKLCH ramps on a fixed lightness curve, hex fallbacks generated
- [ ] Semantic colour tokens for bg, surface, border, text, icon, action, feedback, focus
- [ ] Light and dark mappings with every pairing contrast-tested in CI

Typography ch. 08

- [ ] Role-based scale (display, heading, body, label, caption) as composite tokens
- [ ] Line-heights on the 4px grid; tabular numerals available
- [ ] Responsive strategy that respects zoom and text-spacing overrides

Spacing ch. 09

- [ ] 4px-based scale with guessable token names
- [ ] Semantic component spacing with density modes
- [ ] Target sizes meet WCAG 2.2 AA (24px) and platform touch guidance

Layout ch. 10

- [ ] Named breakpoints, columns, gutters and margins as tokens
- [ ] App shell regions and content-width tokens defined
- [ ] Every pattern planned for 320px reflow

Pipeline ch. 06

- [ ] Single token source (DTCG JSON) compiled to all platforms and the design tool
- [ ] Lint rule against raw values in product code

> ## Tokens first, components second.
>
> Rule #4 — Layer One

With foundations in place, the next layer is components: their APIs, variants, states and accessibility behaviour. That is where Part 03 of this playbook begins.

BACK MATTER

References

Every statistic and normative claim in this playbook links here. Accessed November 2026. Where a source is vendor-published or small-sample, the caveat is stated where it is cited.

1 **Layer One — Design systems studio.** Layer One. layerone.co.in, 2026.
www.layerone.co.in/

2 **Atomic Design.** Brad Frost. Self-published, 2016.
atomicdesign.bradfrost.com/

3 **Design tokens.** Salesforce. Lightning Design System, n.d..
v1.lightningdesignsystem.com/design-tokens/
Salesforce's design system team is widely credited with coining the term 'design tokens' (c. 2014).

4 **Design Tokens specification reaches first stable version.** Design Tokens Community Group. W3C, 2025.
www.w3.org/community/design-tokens/2025/10/28/design-tokens-specification-reaches-first-stable-version/

5 **The Mythical Man-Month: Essays on Software Engineering.** Frederick P. Brooks Jr.. Addison-Wesley, 1975.
Source of the observation that communication paths grow as n(n–1)/2 with team size.

6 **Design Systems Report 2025.** zeroheight. zeroheight, 2025.
zeroheight.com/how-we-document
Survey of 294 respondents (Sep–Nov 2024). 79% report a dedicated design system team; average team size ≈3 (<100 employees), ≈4 (100–499), ≈9 (500+).

7 **Measuring the value of design systems.** Figma. Figma blog, n.d..
www.figma.com/blog/measuring-the-value-of-design-systems/
Vendor-run internal experiment: participants with a relevant, up-to-date design system completed a design objective 34% faster. Sample details are limited; treat as indicative.

8 **Yes, Design Systems Do Improve Developer Efficiency and Design Consistency.** Sparkbox. Sparkbox Foundry, n.d..
sparkbox.com/foundry/design_system_roi_impact_of_design_systems_business_value_carbon_design_system
Eight developers built a form from scratch, then with IBM Carbon: median 4.2 h vs 2 h (≈47% faster). Small sample; the scratch build always came first, so familiarity with the design favours the second build.

9 **The WebAIM Million: The 2025 report on the accessibility of the top 1,000,000 home pages.** WebAIM. Utah State University, 2025.
webaim.org/projects/million/2025
Low-contrast text detected on 79.1% of home pages; 94.8% had at least one detectable WCAG 2 failure.

10 **Apparent usability vs. inherent usability: experimental analysis on the determinants of the apparent usability.** Masaaki Kurosu, Kaori Kashimura. CHI '95 Conference Companion, ACM, 1995.
doi.org/10.1145/223355.223680
Origin of the 'aesthetic-usability effect': visual appeal correlated strongly with perceived ease of use.

11 **How do users evaluate the credibility of Web sites? A study with over 2,500 participants.** B. J. Fogg et al.. Proceedings of DUX 2003, ACM, 2003.
doi.org/10.1145/997078.997097
'Design look' was the most frequently mentioned factor in participants' credibility comments.

12 **Web Content Accessibility Guidelines (WCAG) 2.2.** W3C. W3C Recommendation, 2023.
www.w3.org/TR/WCAG22/

13 **Directive (EU) 2019/882 — European Accessibility Act.** European Parliament and Council. Official Journal of the EU, 2019.
eur-lex.europa.eu/eli/dir/2019/882/oj
Applies to many products and services placed on the EU market from 28 June 2025.

14 **Team Models for Scaling a Design System.** Nathan Curtis. EightShapes, 2015.
eightshapes.com/articles/team-models-for-scaling-a-design-system
Introduces the solitary, centralised and federated team models.

15 **The Salesforce Team Model for Scaling a Design System.** Salesforce UX. Medium, n.d..
medium.com/salesforce-ux/the-salesforce-team-model-for-scaling-a-design-system-d89c2a2d404b
Describes the 'cyclical' hybrid: a central team plus a federated group of contributors.

16 **The Fallacy of Federated Design Systems.** Nathan Curtis. EightShapes, n.d..
nathanacurtis.substack.com/p/the-fallacy-of-federated-design-systems-23b9a9a05542
Argues that successful systems always have a central team, plus federated participation.

17 **Design Tokens Format Module 2025.10.** Design Tokens Community Group. W3C Community Group Final Report, 2025.
www.designtokens.org/tr/2025.10/format/
A Community Group report, not a W3C Standard; widely implemented by token tooling.

18 **Semantic Versioning 2.0.0.** Tom Preston-Werner. semver.org, 2013.
semver.org/

19 **Spacing.** IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/spacing/overview/

20 **Color system: how the system works.** Google. Material Design 3, n.d..
m3.material.io/styles/color/system/how-the-system-works
Tonal palettes generated in the HCT colour space; colour roles mapped from tones.

21 **A perceptual color space for image processing.** Björn Ottosson. bottosson.github.io, 2020.
bottosson.github.io/posts/oklab/
The original definition of Oklab, from which OKLCH is derived.

22 **CSS Color Module Level 4.** W3C CSS Working Group. W3C, n.d..
www.w3.org/TR/css-color-4/

23 **Tailwind CSS v4.0.** Tailwind Labs. tailwindcss.com, 2025.
tailwindcss.com/blog/tailwindcss-v4
The default colour palette moved to OKLCH.

24 **Understanding Success Criterion 1.4.3: Contrast (Minimum)**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

25 **Understanding the scale**. WorkOS. Radix Colors documentation, n.d..
www.radix-ui.com/colors/docs/palette-composition/understanding-the-scale
A 12-step scale where each step has a defined job: backgrounds, component states, borders, solids, text.

26 **Understanding Success Criterion 1.4.11: Non-text Contrast**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/non-text-contrast.html

27 **APCA: Accessible Perceptual Contrast Algorithm**. Myndex Research. git.apcacontrast.com, n.d..
git.apcacontrast.com/documentation/APCA_in_a_Nutshell
A candidate contrast method explored for future WCAG versions. Not normative under WCAG 2.x.

28 **Design tokens**. Atlassian. Atlassian Design System, n.d..
atlassian.design/foundations/tokens/design-tokens

29 **Themes**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/themes/overview/

30 **Design tokens**. Adobe. Spectrum, n.d..
spectrum.adobe.com/page/design-tokens/

31 **Variable fonts guide**. MDN contributors. Mozilla, n.d..
developer.mozilla.org/en-US/docs/Web/CSS/CSS_fonts/Variable_fonts_guide

32 **Type scale tokens**. Google. Material Design 3, n.d..
m3.material.io/styles/typography/type-scale-tokens

33 **Utopia: fluid responsive design**. James Gilyead, Trys Mudford. utopia.fyi, n.d..
utopia.fyi/

34 **Typography**. Apple. Human Interface Guidelines, n.d..
developer.apple.com/design/human-interface-guidelines/typography

35 **Understanding Success Criterion 1.4.12: Text Spacing**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/text-spacing.html

36 **Understanding Success Criterion 1.4.10: Reflow**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/reflow.html

37 **The Elements of Typographic Style**. Robert Bringhurst. Hartley & Marks, 2004.
Recommends 45–75 characters per line for single-column text, with 66 widely regarded as ideal.

38 **Typography: type sets (productive and expressive)**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/typography/type-sets/

39 **Spacing**. Atlassian. Atlassian Design System, n.d..
atlassian.design/foundations/spacing
8px base; tokens from space.025 (2px) to space.1000 (80px), named as multiples of the base.

40 **Designing: structure**. Google. Material Design 3, n.d..
m3.material.io/foundations/designing/structure

41 **Space in Design Systems**. Nathan Curtis. EightShapes, 2016.
Introduces the inset, stack and inline vocabulary for spacing tokens.

42 **Understanding Success Criterion 2.5.8: Target Size (Minimum)**. W3C WAI. W3C, 2023.
www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html

43 **Accessibility**. Apple. Human Interface Guidelines, n.d..
developer.apple.com/design/human-interface-guidelines/accessibility
Recommends a minimum control size of 44×44 pt on iOS/iPadOS.

44 **Layout: breakpoints (window size classes)**. Google. Material Design 3, n.d..
m3.material.io/foundations/layout/breakpoints/overview
Compact < 600dp, medium 600–839dp, expanded 840–1199dp, large 1200–1599dp, extra-large ≥ 1600dp.

45 **2x Grid**. IBM. Carbon Design System, n.d..
www.carbondesignsystem.com/elements/2x-grid/overview/

BACK MATTER

About Layer One and the author

Layer One is a design systems studio in Bangalore, India. We set up the foundations, scale them across products and markets, and put in the governance that keeps teams using them.

Aniket Pandey

Aniket leads Layer One. His design systems work spans three very different kinds of organisation:

- **JPMorgan Chase**, Senior Designer, Design Systems (2025–present): company-wide design patterns, an AI-assisted workflow for drafting them, and tooling that checks design files and code against the system, used by 500+ designers and 1,000+ engineers.
- **CARS24**, Design Systems Lead (2024–25): the design system for 70+ designers and 200+ engineers, a new palette and token architecture for the rebrand, governance, and a pattern library, putting 10+ products in three countries on one token system.
- **IBM**, UX Designer, design systems (2021–24): design systems for IBM clients including Cardinal Health and Riyadh Air, from foundations and tokens to governance, contribution and releases.

Source: layerone.co.in. [1]

The rules we work by

01	A design system is an agreement, not a library
02	Audit before you build
03	Name things so people can guess them
04	Tokens first, components second
05	Accessible by default, not by audit
06	Every decision comes with a reason
07	Build where the system lives: your files, your repo
08	Measure time saved, not usage
09	Docs decide adoption
10	Leave the team able to run it without us

WHERE TO START

Start with a two-week audit.

We map every style, component and token you have in Figma and code, check accessibility, and give you a ranked plan for what to fix first. Fixed scope, fixed price, and credited in full if you go on to a Foundations build.

→ layerone.co.in
→ layerone@proton.me
→ Bangalore, India · working worldwide

What's next in this playbook

This first edition covers strategy and foundations. Later editions extend the same evidence standard to the rest of the system:

PART 03

Components

API design, variants vs props, states, accessibility behaviour, and the first 20 components to build.

PART 04

Patterns and content

Forms, tables, navigation, empty and error states, and UX writing as part of the system.

PART 05

Scaling and tooling

Multi-brand, multi-platform token pipelines, linting, Figma–code sync, and AI in the system workflow.